TunnelEye: An Explainable Framework for Real-Time Detection of Malicious DoH Traffic in High-Speed Backbone Networks

Jiajia Du, Songtao Liu, Cheng Zhi, Junfan Zhao, Hua Wu, *Member, IEEE*, and Guang Cheng, *Senior Member, IEEE*

***Abstract*—Since plaintext DNS queries are vulnerable to eavesdropping and tampering, encrypted DNS protocols such as DoH (DNS over HTTPS) have been widely adopted. However, attackers can exploit DoH by hiding malicious DNS requests and responses in HTTPS traffic, enabling covert communication and evading detection. Existing methods rely on generic bidirectional traffic statistics that lack interpretability, are costly to compute, and may become unavailable or unstable in backbone monitoring nodes with massive unidirectional and sampled traffic. To address these issues, we propose TunnelEye, an explainable real-time framework for malicious DoH detection. It uses protocol-level behavioral analysis to capture DoH features that remain stable across traffic directions and sampling rates, and introduces an efficient data structure, DoH-Sketch, to ensure real-time performance. Experiments on public and self-collected datasets show that TunnelEye enables second-level detection with high accuracy. On public datasets, it achieves 99.88% accuracy. In backbone networks with sampled traffic, TunnelEye outperforms existing methods, improving accuracy by 14.5% at a 1/128 sampling rate and reducing the false positive rate by 31% under interference from background traffic.**

***Index Terms*—DNS over HTTPS, malicious DoH traffic detection, explainability, backbone network, packet sampling.**

I. Introduction

DOMAIN Name System (DNS) is a critical fundamental infrastructure in Internet architecture, enabling the mutual mapping of domain names and IP addresses. However, the DNS protocol transmits data in plaintext, which has long posed security risks [1]. Attackers can easily intercept, alter, or forge DNS queries to steal sensitive information or attack maliciously [2], [3], [4]. Moreover, due to DNS traffic's critical role in network operations, security mechanisms such as firewalls typically allow DNS traffic to pass through unimpeded, creating an opportunity for malicious DNS traffic (such as covert data transfers via DNS tunneling) to bypass conventional security defenses. 90% of companies had encountered DNS attacks, resulting in significant financial losses [5]. Roughly one-third of DNS attacks were associated with DNS tunneling [6].

To enhance the security of the DNS protocol, the IETF (Internet Engineering Task Force) introduced a series of encrypted DNS protocols [7], [8], [9], among which DNS over HTTPS (DoH) [8] proposed in 2018 encrypts DNS traffic via the HTTPS protocol. With the broad support of HTTPS across existing web infrastructure, DoH quickly became the most widely deployed encrypted DNS protocol. Globally, more than 40 DNS service providers support DoH, including major providers like Cloudflare, Google, and Quad9. On the client side, popular browsers like Firefox [10] and Chrome [11] offer official secure DNS services by configuring DoH. By encrypting DNS requests and responses, DoH effectively prevents man-in-the-middle attacks, traffic eavesdropping, and DNS poisoning, thereby enhancing the privacy and security of DNS communications.

However, the widespread adoption of DoH also introduces new security challenges. The encryption of DoH traffic prevents direct DNS inspection. Attackers can abuse DoH as a covert tunnel, known as a DoH tunneling attack. In such attacks, attackers create and control DoH tunnels. They encapsulate non-DNS payloads (like C2 commands or exfiltrated data) within crafted DNS queries and responses. This enables malicious goals such as command-and-control (C2) and data exfiltration. In this paper, we focus on detecting malicious DoH traffic generated by DoH tunneling attacks. Due to the encryption of DoH, traditional malicious DNS tunneling detection methods [12], [13] that rely on plaintext information become ineffective.

Existing research [14], [15], [16] has proposed several methods for detecting malicious DoH traffic. Most of these methods require grouping packets into flows and extracting complex statistical features, which results in inefficient detection in high-speed backbone networks. Although these methods have been validated in Local Area Networks (LANs), backbone

Received 2 July 2025; revised 16 March 2026; accepted 4 April 2026. Date of publication 10 April 2026; date of current version 17 April 2026. This work was supported by National Natural Science Foundation of China under Grant 62572117. The associate editor coordinating the review of this article and approving it for publication was D. Naboulsi. *(Corresponding author: Hua Wu.)*

Jiajia Du, Songtao Liu, Cheng Zhi, and Junfan Zhao are with the School of Cyber Science and Engineering, Southeast University, Nanjing 210096, China (e-mail: jjdu@seu.edu.cn; stliu@seu.edu.cn; czhi@seu.edu.cn; jfzhao@seu.edu.cn).

Hua Wu is with the School of Cyber Science and Engineering, Southeast University, Nanjing 210096, China, and also with the Purple Mountain Laboratories for Network and Communication Security, Nanjing 210023, China (e-mail: hwu@seu.edu.cn).

Guang Cheng is with the School of Cyber Science and Engineering, Southeast University, Nanjing 210096, China, and also with Jiangsu Province Engineering Research Center of Security for Ubiquitous Network, Jiangsu, Nanjing 211189, China (e-mail: chengguang@seu.edu.cn).

Digital Object Identifier 10.1109/TNSM.2026.3682686

1932-4537 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 22,2026 at 05:59:52 UTC from IEEE Xplore. Restrictions apply.

networks handle significantly larger traffic volumes. Backbone networks typically use sampling to reduce data size, yet most existing methods rely on extracting statistical features from unsampled full traffic. In addition, asymmetric routing is widely deployed in backbone networks, where requests and responses traverse different paths, resulting in unidirectional traffic at monitoring points. None of the existing methods have considered the impact of unidirectional traffic on features.

Therefore, detecting malicious DoH in high-speed backbone networks presents the following challenges: 1) Most existing methods require grouping packets into flows and extracting features through resource-intensive operations such as packet sorting and variance calculations. Consequently, these methods are only suitable for small-scale networks and fail to promptly process massive data in backbone networks, making real-time detection difficult. 2) To address the pressure caused by massive data in high-speed backbone networks, network administrators typically employ sampling to reduce the data volume for analysis. However, the available information on malicious DoH traffic is also diminished during sampling, making it more concealed. 3) In backbone networks, requests and responses usually traverse different paths due to the widespread deployment of asymmetric routing, resulting in monitoring points collecting only unidirectional traffic, while existing detection methods are designed based on the assumption of bidirectional traffic. We propose TunnelEye,[1] a real-time malicious DoH traffic detection framework tailored to address these challenges for backbone networks. The main contributions of this paper are as follows:

- To address the significant challenges of asymmetric routing and traffic sampling in backbone networks, we construct a unidirectional traffic feature set applicable to sampled traffic based on an in-depth analysis of DoH traffic at the protocol level. This feature set provides precise and highly explainable features for malicious DoH traffic detection.
- To process massive data in real-time within high-speed backbone networks, we design the data structure DoH-Sketch for traffic feature extraction. DoH-Sketch does not require grouping packets into flows, significantly reducing computational and storage overhead and improving feature extraction efficiency.
- We conduct extensive training and testing on both public datasets and self-collected datasets captured at the 10 Gbps campus network boundary. Experimental results demonstrate that TunnelEye can detect malicious DoH traffic rapidly and accurately, achieving a feature extraction speed of 35.17 Gbps and enabling attack detection at the second level. Compared to existing methods, TunnelEye improves the average accuracy by 14.5% at a 1/128 sampling rate and reduces the false positive rate by 31% under real-world network background traffic, exhibiting stronger robustness.

[1]The codes are available at https://github.com/iaiapetrk/TunnelEye

II. Related Work

In this section, we review related work from three perspectives: DNS traffic analysis, encrypted DNS analysis, and malicious DoH detection.

A. DNS Traffic Analysis

Research on DNS traffic analysis primarily encompasses DNS measurement and malicious activity detection.

DNS measurement studies investigate various aspects of DNS systems, including operational efficiency, system architecture, asset mapping, and security analysis. Pinto et al. [17] pointed out that implementation differences among DNS measurement tools can lead to biased performance results. The tool Trufflehunter [18] successfully inferred the cache architectures of public DNS services such as Google and Cloudflare through TTL timeline analysis. Li et al. [19] proposed the DNSScope framework, enabling fine-grained estimation of query arrival rates at recursive resolvers, achieving an average absolute error of 0.124. Moura et al. [20] revealed the problem of "DNS polarization" through passive TCP-based measurements. Lyu et al. [21] automatically identified and monitored DNS assets and their security health status in enterprise networks by passively analyzing DNS traffic.

Many studies aim to identify various malicious behaviors from DNS traffic. Detection for service availability mainly focuses on DNS flood and amplification DDoS threats. Şahin et al. [22] proposed the HELP4DNS system, which leverages a programmable data plane to implement DNS query rate limiting and firewalling on P4 switches, effectively defending against flooding and amplification attacks. Detection for data integrity focuses on tampering attacks such as DNS hijacking and cache poisoning. Afek et al. [23] proposed the POPS system to detect DNS cache poisoning attacks, reducing the attack success probability to 0.0076%. Detection of covert channels covers threats such as DNS tunneling and DGA domains. Wang et al. [1] provided a comprehensive survey of DNS tunneling detection. Manasrah et al. [24] proposed a machine learning method based on lexical features of DNS traffic to efficiently detect DGA-generated malicious domain names.

B. Encrypted DNS Analysis

Plaintext DNS communication between clients and resolvers has long faced risks of privacy eavesdropping and man-in-the-middle (MITM) attacks. To provide transport confidentiality, the IETF standardized three encrypted DNS protocols, including DNS over TLS (DoT) published in 2016 [7], DNS over HTTPS (DoH) standardized in 2018 [8], and DNS over QUIC (DoQ) standardized in 2022 [9].

Among the standardized encrypted DNS protocols, DoH has become the most popular due to broad support from DNS resolution service providers and clients. Currently, major public DNS resolvers (e.g., Cloudflare, Google, and Quad9) support both DoH and DoT. On the client side, browsers such as Chrome [11], Firefox [10], and Edge [11] use DoH rather than DoT as the encrypted DNS solution. In addition, Apple

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 22,2026 at 05:59:52 UTC from IEEE Xplore. Restrictions apply.

announced in June 2020 that macOS 11 and iOS 14 natively support DoT and DoH.

Several studies have examined the reachability, performance, and security of DoH. Lu et al. [25] confirmed through large-scale measurements that DoH has achieved global reachability and that its adoption continues to grow. Borgolte et al. [26] similarly found that the number of DoH-supporting providers is steadily increasing. Regarding performance, Borgolte et al. [26] systematically measured DoH performance and pointed out that the TLS handshake is the main bottleneck. Böttger et al. [27] verified that DoH has limited impact on user experience, whereas Deccio et al. [28] noted that deployment of its key optimization mechanism, TCP Fast Open (TFO), remains insufficient and may constrain performance.

With respect to DoH security, existing studies indicate that while DoH improves privacy, it also introduces new attack surfaces. Hu et al. [29] found that even after encryption, packet inter-arrival times of DoH, DoT, and DoQ may still leak information. To mitigate such traffic analysis attacks, RFC 8467 recommends padding messages before encryption; however, Bushart et al. [30] showed that padding alone is insufficient. Their method can infer websites visited by users from encrypted DNS traffic with an accuracy of 32.0%. Niakanlahiji et al. [31] further proposed a client-side obfuscation approach that combines compressive-sensing-based padding, dummy query injection, and random delays, reducing the accuracy of traffic analysis attacks from 95.58% to below 9%. Lyu et al. [32] systematically summarized the technical evolution of encrypted DNS and pointed out that malware is increasingly abusing DoH for covert communications, posing new challenges to detection techniques. Therefore, while DoH enhances privacy, it also brings new security challenges, calling for further research on attack detection and defense.

C. Malicious DoH Detection

DoH tunneling is one of the most common forms of malicious DoH traffic. Consequently, many studies on malicious DoH traffic focus on detecting DoH tunneling activities. Existing research on malicious DoH detection can be broadly categorized into two types: rule-matching-based methods and artificial intelligence (AI)-model-based methods, with some approaches combining both techniques.

1) Rule-Matching-Based Methods: These methods detect malicious DoH traffic using predefined rules such as thresholds or allowlist. Kwan et al. [14] analyzed packet size, packet rate, and throughput to determine thresholds for distinguishing malicious DoH traffic from benign DoH traffic. Zhan et al. [33] proposed a allowlist mechanism using TLS fingerprints However, attackers can evade detection by mimicking popular TLS implementations. Additionally, Moure-Garrido et al. [34] focused on real-time detection and proposed a real-time detection system for malicious DoH traffic based on packet size and inter-packet interval, but it detected only about 3% of dns2tcp connections.

Although rule-based methods are computationally efficient, they rely on simple features and fixed thresholds, which are sensitive to traffic sampling and asymmetric routing common in high-speed backbone networks. These factors often lead to unreliable features, limiting detection accuracy and making it difficult to identify complex malicious DoH behaviors in backbone networks.

2) AI-Model-Based Methods: These methods can be categorized into deep learning-based and machine learning-based approaches.

Deep Learning (DL)-Based Methods:

DL-based methods often rely on primary traffic features and learn complex representations through neural networks. Ding et al. [35] proposed a variational autoencoder-based anomaly detection model using sequences constructed from packet length, direction, and inter-arrival time. However, it requires reconstructing bidirectional flows, which is difficult in high-speed backbone networks. Liu et al. [36] combined Model-Agnostic Meta-Learning (MAML) with CNNs to improve detection in low-data scenarios, but still relied on bidirectional flow-level features. Wu et al. [37] applied GANs to generate synthetic DoH traffic on small datasets, improving detection accuracy. However, it assumes complete bidirectional flow capture, which is difficult in backbone networks. Du et al. [38] proposed a semi-supervised autoencoder model, reducing the need for labeled data but still relying on bidirectional flow analysis. Jiao et al. [39] introduced an attention-based CNN model for IPv6 DoH traffic, which uses flow-level statistics and ignores unidirectional flow effects.

Most DL-based malicious DoH detection methods leverage deep learning's automatic feature extraction but typically rely on flow-level features. This flow reconstruction overhead limits real-time deployment in high-speed backbone networks. Moreover, most methods assume unsampled and symmetric traffic, leading to significant performance degradation in backbone networks. DL models require large datasets and lack explainability.

Machine Learning (ML)-Based Methods: ML-based methods are more explainable and require smaller datasets for training. As a result, they are widely adopted in practical detection tasks. Zebin et al. [15] used 29 statistical features extracted from full traffic flows in the CIRA-CIC-DoHBrw-2020 [40] dataset to train a stacked Random Forest model. Some features are unavailable in unidirectional flows, while others become unreliable when traffic is sampled. Niktabe et al. [41] improved transparency using features based on mutual information and correlation, but still relies on bidirectional sequences. Wang et al. [42] combined threshold rules with ML models for malicious DoH detection, but requires complete bidirectional flows. Aggarwal et al. [16] and Zhan et al. [33] refined feature sets and classifiers, but also depend on flow-level data derived from unsampled, bidirectional traffic. Mitsuhashi et al. [43], [44] proposed a hierarchical ML framework using 28 flow-level features for DoH tunneling tool detection, evaluated only under bidirectional flow conditions. Qiu et al. [45] partially considered unidirectional traffic in experiments, but the dataset contained only nine malicious samples, leaving the method's effectiveness in backbone networks unverified. Qiu et al. [45] partially considered unidirectional traffic, but evaluated only on a very small malicious dataset.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 22,2026 at 05:59:52 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF EXISTING MALICIOUS DOH DETECTION METHODS

Methods		References	Tunnel Tool Diversity	High Accuracy	No flow grouping	Sampling	Asymmetric Routing	Real-time
rule-matching-based		[8]				✓	✓	✓
		[11]		✓				✓
		[12]					✓	✓
AI-model-based	DL	[13],[16]		✓				✓
		[14]	✓					
		[15]		✓				
		[33]	✓	✓				
	ML	[9]-[11],[34]-[36]		✓				
		[37],[38]	✓	✓				
		our method	✓	✓	✓	✓	✓	✓

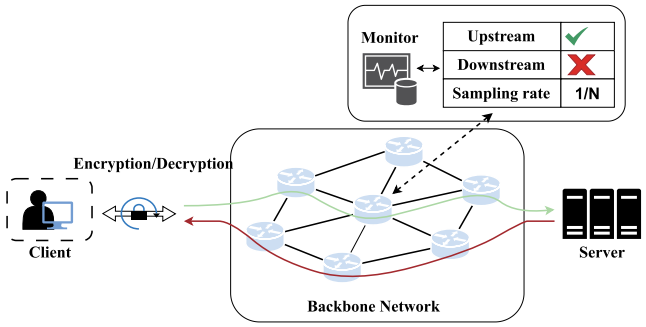


Fig. 1. Backbone network monitoring environment.

Overall, most ML-based methods rely on flow reconstruction and numerous statistical features, leading to high computational overhead. Additionally, existing methods typically assume bidirectional flows and unsampled traffic. In backbone networks, however, asymmetric routing often results in many unidirectional flows, which can cause feature loss or distortion, while sampling further limits their applicability.

3) Summary: In general, existing malicious DoH traffic detection methods have the following limitations. Rule-based methods are inaccurate in identifying complex malicious traffic and are susceptible to evasion. Most AI-based methods are inefficient, making them unsuitable for real-time detection. More critically, most methods assume bidirectional traffic and do not support sampled traffic, limiting their applicability in backbone networks. The characteristics of the existing malicious DoH detection methods are summarized in Table I.

In this paper, we propose TunnelEye, a framework designed to detect malicious DoH traffic in high-speed backbone networks. TunnelEye leverages a feature set specifically constructed for sampled unidirectional traffic and employs our custom-designed fast data structure, DoH-Sketch, to enable real-time detection.

III. PROBLEM BACKGROUND

Deploying malicious DoH traffic detectors in high-speed backbone networks faces numerous practical constraints. As shown in Fig. 1, monitors are placed at backbone nodes to passively observe traversing traffic. We first outline the constraints of the backbone monitoring environment and then distill the main challenges faced when deploying malicious DoH detectors in practice.

A. Constraints on the Backbone Network Monitoring

In backbone network monitoring, two categories of constraints significantly impact the deployability of malicious DoH detectors:

1) Asymmetric Routing: Asymmetric routing is ubiquitous in backbone networks. As illustrated in Fig. 1, packets belonging to the same end-to-end connection often traverse different paths in the forward and reverse directions. Consequently, a monitor may only observe unidirectional traffic. Statistics from two 10 Gbps Internet exchange link datasets (MAWI-202006031400 and MAWI-202006101400) from the WIDE project in Japan indicate that the proportions of unidirectional flows are 84.10% and 81.55% respectively [46]. This demonstrates the widespread existence of asymmetric routing in the WIDE backbone, rendering features that rely on bidirectional interaction semantics incomputable or distorted.

2) Traffic Sampling: To constrain storage and processing overheads, monitoring edges typically implement packet sampling at fixed ratios. Sampling reduces the number of observed packets and may eliminate burstiness or short-term behavioral patterns. Therefore, feature extraction methods relying on fine-grained packet sequences, inter-arrival times, or complete byte sequences may suffer from severe distortion after sampling.

B. Deployment Challenges

Although existing malicious DoH detection methods perform well in laboratory or LAN environments, they typically assume access to unsampled, complete bidirectional flow data. However, directly migrating these methods to the backbone monitoring environment faces the following challenges:

1) Reliance on Bidirectional Flow Features: Existing methods widely utilize bidirectional statistical features. However, asymmetric routing in backbone networks results in monitors typically having only unidirectional visibility. This implies that the bidirectional features relied upon by existing methods become incomputable or distorted. Adopting these features in backbone networks leads to semantic errors, thereby causing a drastic degradation in model accuracy.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 22,2026 at 05:59:52 UTC from IEEE Xplore. Restrictions apply.

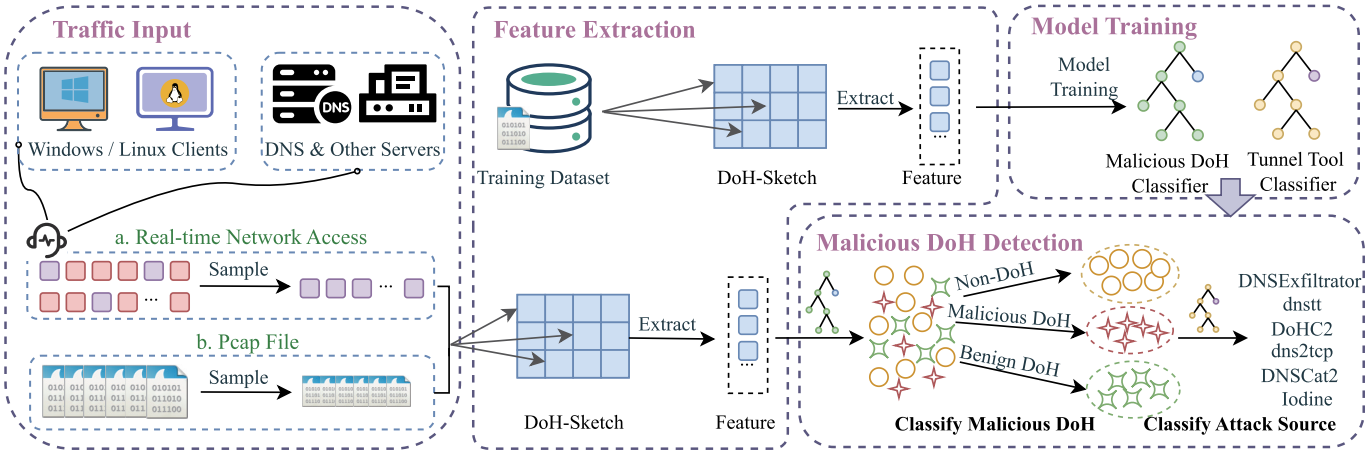


Fig. 2. The framework of TunnelEye.

2) Feature Distortion Caused by Sampling: To cope with massive network traffic, employing sampling strategies is the norm for backbone detectors. However, existing flow features often rely on fine-grained packet sequence information. Sampling disrupts the temporal structure and sequence integrity of packets, causing these statistical features to fluctuate severely.

3) Performance Bottlenecks: Existing methods typically require maintaining per-flow states to perform flow reassembly, sorting, and statistical computation. Under the pressure of millions of concurrent connections and massive traffic volumes in backbone networks, maintaining huge flow tables consumes enormous memory, and frequent table lookups and updates lead to severe latency. In summary, detecting malicious DoH traffic in backbone networks calls for a new detection scheme. Such a scheme should offer unidirectional compatibility and robustness to sampling to cope with real-world network conditions, adopt a lightweight design to enable high-throughput online analysis.

IV. Methodology

To achieve accurate and real-time detection of malicious DoH traffic in high-speed backbone networks, we propose TunnelEye. This framework comprises four core modules: traffic input, feature extraction, model training, and malicious DoH detection, as illustrated in Fig. 2. TunnelEye is based on the unidirectional traffic feature set we constructed, which is tailored for sampled traffic. In the following subsections, we describe the constructed feature set and each module within TunnelEye.

A. Unidirectional Traffic Feature Set Construction

To construct features with higher explainability and precision, we analyze encrypted DoH traffic by examining the transmission pattern, working mechanism, and distinctions between benign and malicious DoH traffic.

Transmission patterns.

In HTTPS communication, application data are encapsulated within HTTP/2 frames, encrypted at the TLS layer, and then fragmented into TLS records. Each TLS record is prefixed with a header (Fig. 3a). TLS records may be encapsulated within packets in two distinct ways according to the size of the TLS record. When the size is small and does not exceed the TCP transmission limit, such as DoH traffic, the entire TLS record can be transmitted within a single packet, as shown in Fig. 3a. Conversely, in most HTTPS scenarios such as web browsing, video streaming, and social media, the size of TLS record is often large enough to exceed the TCP transmission limit. As a result, a single TLS record may be fragmented across multiple TCP segments, leading to an entire TLS record within multiple packets, as shown in Fig. 3b. We refer to this phenomenon as cross-packet transmission. Overall, DoH traffic follows the pattern shown in Fig. 3a, whereas other HTTPS traffic corresponds to Fig. 3b.

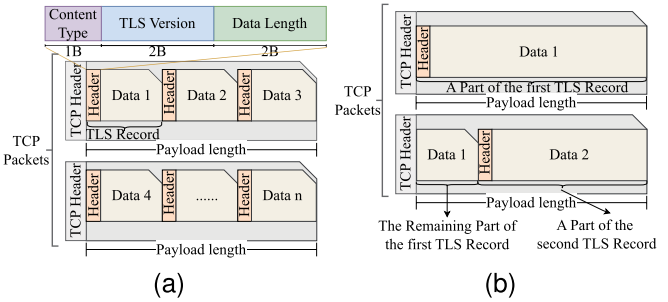


Fig. 3. TLS Record Encapsulation over TCP Segments (a) One TLS record is encapsulated within a single TCP segment. (b) One TLS record is fragmented across multiple TCP segments.

Working mechanism. Modern web applications, which extensively utilize RESTful architecture and Content Delivery Networks (CDNs), often require browsers to initiate multiple DNS queries to globally distributed servers. This behavior imposes high demands on the efficiency of domain name resolution. To address the head-of-line blocking issue inherent in HTTP/1.1, most DoH servers adopt HTTP/2, which supports multiplexing and allows concurrent transmission of multiple DNS queries over a single connection. Multiple DNS requests/responses are coalesced into a single packet. These behaviors result in multiple TLS records appearing within the

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 22,2026 at 05:59:52 UTC from IEEE Xplore. Restrictions apply.

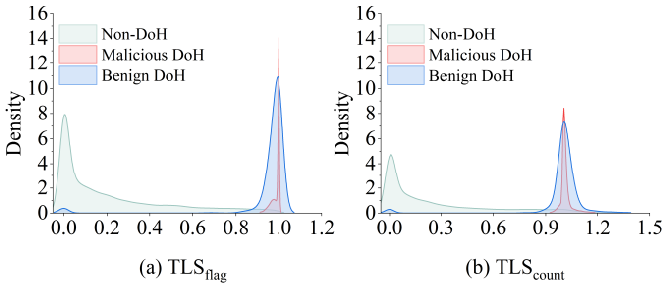


Fig. 4. Kernel density estimation of TLS_{count} and TLS_{flag}.

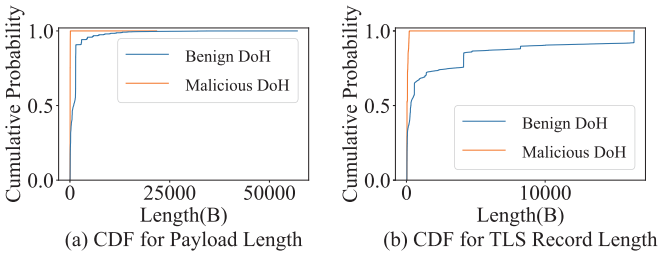


Fig. 5. Cumulative distribution functions (CDFs) of L_{PL} and L_{TLS}.

same packet. Distinctions between benign and malicious DoH traffic. Due to differences in data volume between malicious and benign DoH traffic, packet length has been widely used for detecting malicious DoH traffic [35], [39]. In benign DoH connections, clients and DoH servers exchange a significant amount of SSL/TLS key exchange data. In contrast, malicious DoH traffic (e.g., DoH covert channels) often transmits smaller heartbeat packets to maintain long-term connections. Additionally, previous research on malicious DNS tunnel detection has shown that query domain length is also an important feature. However, due to the encrypted nature of DoH traffic, domain length cannot be directly obtained. Some studies [33] use TLS record length (L_{TLS}) as a substitute for domain length. Considering these distinctions, we have constructed two features related to length distribution to better differentiate between benign and malicious DoH traffic.

To capture these characteristics revealed in the above analysis, we construct four features designed to improve the accuracy and explainability of malicious DoH traffic detection.

Feature 1: *The ratio of packets with TLS record at the beginning of payload.* HTTPS data from other services often exhibits cross-packet transmission, which causes multiple TLS records to appear not at the beginning of the packet, but at the end of the previous TLS record. Therefore, in DoH traffic packets, the TLS record header generally resides at the beginning of the payload, and the ratio of such packets is significantly high. We define this ratio as a feature denoted as TLS_{flag}.

Feature 2: *TLS record density in packets.* The multiplexing mechanism of HTTP/2 often results in multiple TLS records being encapsulated within a single packet in DoH traffic. Accordingly, we define the ratio of the number of TLS records in the payload to the total number of packets as a feature, denoted as TLS_{count}.

We analyze the distribution of TLS_{flag} and TLS_{count} using kernel smoothing. Kernel smoothing applies a kernel function (e.g., Gaussian kernel) to weight and smooth the data, effectively eliminating noise from the original data and revealing the trends and details of its distribution. The Kernel Density Estimation (KDE) plots are shown in Fig. 4, where the y-axis represents the probability density, indicating the relative probability density distribution of the data across different intervals. As illustrated in Fig. 4, non-DoH traffic and DoH traffic exhibit significant differences in the distribution of these two features. Compared to non-DoH traffic, DoH traffic contains more TLS records and experiences fewer instances of cross-packet transmission. Furthermore, within DoH traffic, malicious traffic demonstrates a more concentrated distribution in both TLS_{flag} and TLS_{count} than benign traffic. These distributional differences across different types of traffic indicate that these features significantly enhance the ability to identify DoH traffic. Combined with other features, it can further identify malicious traffic from DoH traffic.

Feature 3: *Payload Length Distribution.* Considering that the payload of a packet carries the actual application data, the payload length (L_{PL}) better reflects the transmitted data volume than the overall packet length. Therefore, we define the distribution of L_{PL} as a feature, denoted as $L_{PL}_scope_N$, which records the ratio of L_{PL} falling within the N-th ΔL.

Feature 4: *TLS Record Length Distribution.* In benign DoH traffic, resolution requests for different domains may involve varying domain lengths and response data, potentially including large resource records, resulting in a wide distribution of L_{TLS}. In contrast, malicious DoH traffic employs restricted communication mechanisms with fixed patterns, leading to lower communication loads and a more uniform, smaller L_{TLS} distribution. Based on this distinction, we define the distribution of L_{TLS} as a feature, denoted as $L_{TLS}_scope_N$, which records the proportion of L_{TLS} falling within the N-th ΔL interval.

We compared the L_{TLS} and L_{PL} distributions between malicious and benign DoH traffic and plotted their cumulative distribution functions (CDFs), as shown in Fig. 5.

Significant differences exist in the L_{TLS} and L_{PL} distributions between malicious and benign DoH traffic. Compared to benign DoH traffic, the L_{TLS} and L_{PL} distributions of malicious DoH traffic are notably narrower, with most values concentrated within a small range (primarily within the $0 \sim 1600$B). Therefore, we calculate $L_{PL}_scope_N$ and $L_{TLS}_scope_N$ based on the first 1600B. Furthermore, the CDF curve for L_{TLS} in malicious DoH traffic is steeper. For benign DoH traffic, it is relatively flat, reflecting the relatively uniform distribution of L_{TLS} in malicious DoH traffic.

In summary, the unidirectional traffic feature set is shown in Table II. These features are specifically designed to reflect the characteristics of DoH (DNS over HTTPS) traffic, which ensures their explainability. Furthermore, since the features are defined as proportion-based metrics, they maintain stability under sampling conditions.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 22,2026 at 05:59:52 UTC from IEEE Xplore. Restrictions apply.

TABLE II
FEATURE SET

Name	Direction	Description
PL	fwd/bwd	Ratio of packets containing payload to total packets.
TLS_{flag}	fwd/bwd	Ratio of packets with TLS record headers at the payload beginning to total packets.
TLS_{count}	fwd/bwd	Ratio of TLS record count in the payload to total packets.
TLS_{count} / TLS_{flag}	fwd/bwd	Ratio of TLS_{count} to TLS_{flag}.
L_{PL}_scope_N	fwd/bwd	Proportion of L_{PL} falling within the N-th ΔL, $N = 1 \sim \lceil 1600B/\Delta L \rceil$.
L_{TLS}_scope_N	fwd/bwd	Proportion of L_{TLS} falling within the N-th ΔL, $N = 1 \sim \lceil 1600B/\Delta L \rceil$.

fwd: the direction of upstream traffic at the monitoring point.
bwd: the direction of downstream traffic at the monitoring point.
ΔL: the fixed-width length interval. Its specific value was determined through experiments in Section Section V..

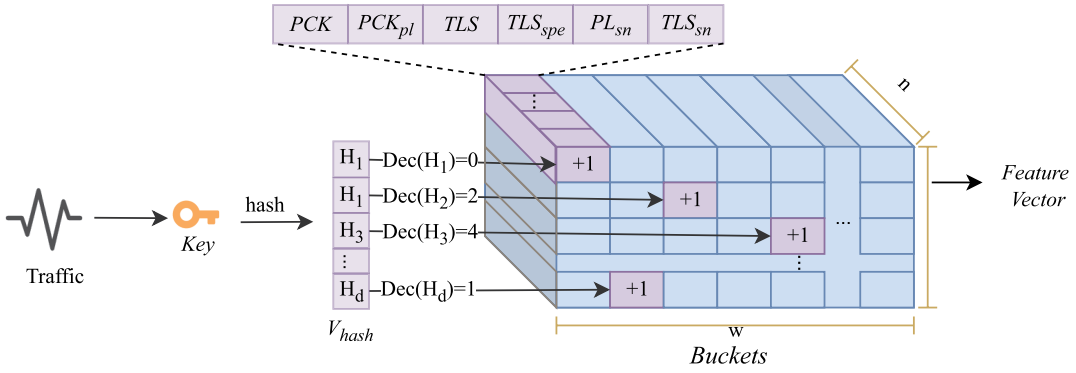


Fig. 6. The structure of DoH-Sketch.

B. Traffic Input

TunnelEye supports two input modes for different usage scenarios in the traffic input module. During model training, it reads traffic from offline pcap files. For real-time detection, it supports analyzing traffic obtained from a traffic collector connected to the backbone network. Both input modes support traffic sampling, and the sampling rate can be configured according to network conditions.

C. Feature Extraction

In the feature extraction module, to quickly extract the proposed features in high-speed backbone networks, we designed DoH-Sketch based on Sketch [46] as a feature extractor. Unlike traditional flow-based feature extraction methods used in prior research, our DoH-Sketch employs a stream-processing algorithm that does not require grouping packets into flows, thus avoiding the storage of flow-state data in memory. Additionally, flow-based approaches often suffer from reduced information availability in sampling scenarios. To address this limitation, our DoH-Sketch adopts a threshold-based cumulative feature extraction strategy.

Design of DoH-Sketch. As shown in Fig. 6, the data structure of DoH-Sketch is a two-dimensional matrix of buckets, denoted by *Buckets*, consisting of $d \cdot w$ buckets. Each bucket contains counters used to calculate features, allowing the simultaneous recording of multiple traffic characteristics of the data packets. The detailed information of the counters in the bucket is shown in the Table III.

We define network traffic as a sequence of m packets arriving sequentially within a time interval, represented as $I = p_0, p_1, \ldots, p_t, \ldots, p_{m-1}$. When a packet arrives, DoH-Sketch extracts its *Key* as the input item, which is defined as $Key = \langle IP_{\text{source}}, IP_{\text{destination}}, 443, TCP \rangle$, where IP_{source} and $IP_{\text{destination}}$ represent the source IP address and destination IP address, respectively. Since malicious DoH detection targets HTTPS traffic, DoH-Sketch handles only traffic where the server port is 443 and the protocol is TCP.

TABLE III
DETAILED DESCRIPTION OF COUNTERS

Name	Type	Description
PCK	Counter	Record all packets
PCK_{pl}	Counter	Record packets containing payload
TLS	Counter	Record TLS records
TLS_{spe}	Counter	Record TLS records at the payload beginning
PL_{sn}	Counter Group	Record packets with L_{PL} falling within the N-th ΔL
TLS_{sn}	Counter Group	Record packets with L_{TLS} falling within in the N-th ΔL

Operations of DoH-Sketch. DoH-Sketch supports two operations: *Update* and *Extract*. The *Update* operation continuously and updates the specific *Buckets* (Algorithm 1), monitoring the changes in real-time. Once the predefined condition is met, the *Extract* operation is automatically triggered to extract feature vectors (Algorithm 2).

- ***Update*:** The *Update* operation maps a packet to the corresponding bucket based on its *Key* and updates the counters in the bucket. Unlike the traditional Sketch, which employs multiple hash functions for mapping, DoH-Sketch uses only a single hash function to

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 22,2026 at 05:59:52 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 *Update*

Input: Packet p_t
Output: *Buckets*
$Key \leftarrow$ ExtractKey(p_t)
$V_{hash} \leftarrow$ FarmHash(Key)
$N \leftarrow \lceil 1600/\Delta L \rceil$
if $p_t.payload$ exists **then**
  $Payload \leftarrow Payload + 1$
  if $p_t.payload.length$ in range N **then**
    $L_{PL}_scope_N \leftarrow L_{PL}_scope_N + 1$
  end if
end if
if IsEasyTLS($p_t.payload$) **then**
  $Easy_TLS \leftarrow Easy_TLS + 1$
  $num_tls \leftarrow num_tls +$ tls_records.count
  for tls_record in tls_records **do**
    if tls_record.length in range N **then**
      $L_{TLS}_scope_N \leftarrow L_{TLS}_scope_N + 1$
    end if
  end for
end if
$otherCounters \leftarrow \{L_{PL}_scope_N, L_{TLS}_scope_N, Easy_TLS, num_tls, Payload\}$
for x in $[1, d]$ **do**
  $P_x \leftarrow$ ExtractPartOfHash(V_{hash}, x)
  $D(P_x) \leftarrow$ CalculateDecimalValue(P_x)
  $bucket \leftarrow$ *Buckets* $[x][D(P_x)]$
  $bucket.Packet \leftarrow bucket.Packet + 1$
  $bucket.otherCounters \leftarrow bucket.otherCounters + otherCounters$
end for
return *Buckets*

Algorithm 2 *Extract*

Input: Packet Key, θ
Output: *Feature_Vector*
$V_{hash} \leftarrow$ FarmHash(Key)
$bucket_indices \leftarrow []$
$Saturation \leftarrow []$
for $x \in [1, d]$ **do**
  $P_x \leftarrow$ ExtractPartOfHash(V_{hash}, x)
  $D(P_x) \leftarrow$ CalculateDecimalValue(P_x)
  $bucket_indices$.append($< x, D(P_x) >$)
  $Saturation$.append(*Buckets*$[x][D(P_x)].Packet$)
  if $Buckets[x][D(P_x)].Packet = \min(Saturation)$ **then**
    $Min_buckets \leftarrow Buckets[x][D(P_x)]$
  end if
end for
if $\min(Saturation) < \theta$ **then**
  return:
end if
$feature \leftarrow$ CalculateFeature($Min_buckets.Counters$)
$Feature_Vector$.append($feature$)
for $b \in bucket_indices$ **do**
  $Buckets[b].Counters \leftarrow Buckets[b].Counters - Min_buckets.Counters$
end for
return: $Feature_Vector$

determine the storage location of a packet through one hash computation. First, DoH-Sketch applies the FarmHash algorithm to hash the packet's *Key*, generating a 128-bit hash value V_{hash}. Then, the first α bytes of V_{hash} are equally divided into d parts, each of length l bits ($\alpha \leq 128$, $w = 2^l$, $22 \leq l \leq 32$). The x-th part is denoted as H_x. By calculating the decimal value $Dec(H_x)$ of H_x, DoH-Sketch locates the specific bucket at $Buckets[x][Dec(H_x)]$.

***Extract*:** The *Extract* operation returns a feature vector composed of the current *Key* and its corresponding cumulative features, with the cumulative features calculated from the values of the counters within the bucket. Like typical sketches, DoH-Sketch performs only accumulation when updating the *Buckets*, while feature extraction is triggered when the PCK counter reaches a specific threshold. To clarify when the *Extract* operation occurs, we define the *saturation* of a bucket as the number of packets mapped to that bucket, equivalent to the PCK counter value. Due to potential partial hash collisions for different *Keys*, the d buckets corresponding to a *Key* may have different saturations. When the minimum saturation among the d buckets reaches the predefined threshold θ, DoH-Sketch extracts the counters from the bucket with the minimum saturation to compute the features. This approach effectively mitigates the impact of hash collisions. After an *Extract* operation, the counters in all d buckets corresponding to the *Key* are decremented by the values of the counters in the bucket with the minimum saturation, preparing for the next *Extract*.

D. Model Training

In this module, we train detection models using the features extracted from pcap files by DoH-Sketch. The detection model consists of two classifiers: the malicious DoH classifier, which identifies malicious DoH traffic from network traffic, and the tunnel tool classifier, which further categorizes the detected malicious traffic based on different attack sources (malicious tunnel tools).

When selecting a model for classifiers to perform detection tasks on high-speed backbone networks, we consider both accuracy and performance. In Section V, we compared various artificial intelligence models to determine the most suitable one. During training, we applied regularization, subsampling, and early stopping to control model complexity and prevent overfitting. To minimize the impact of anomalous data, we used ten-fold cross-validation to train and test all candidate models on the publicly available CIRA-CIC-DoHBrw-2020 dataset and our DoH-Extended datasets. The model showing the best overall accuracy and speed was then chosen.

E. Malicious DoH Detection

In the malicious DoH detection module, the malicious DoH classifier and the tunnel tool classifier, obtained from the

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 22,2026 at 05:59:52 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
DETATILS OF DATASETS

DataSet	Label	Data Size	Source
CIRA-CIC-DoHBrw-2020	Non-DoH	56.1GB	Accessing Web pages
	Benign DoH		AdGuard, Cloudflare, Google DNS, Quad9
		2.35GB	dns2tcp
	Malicious DoH	3.69GB	DNSCat2
		5.39GB	iodine
DoH-Extended	Non-DoH	17.14G	Accessing Web pages
	Benign DoH	27.98G	Quad and Google DNS (from CIRA-CIC-DoHBrw-2020)
		3.15G	AliDNS, DNSPod, 360, OpenDNS
		11.7GB	DoHC2
	Malicious DoH	11.3GB	dnstt
		289MB	DNSExfiltrator
Trace A	Daily Traffic	1.25TB	the WIDE backbone networK
Trace B	Daily Traffic	112GB	the boundary of a campus network

model training module, can be deployed on the backbone network. First, DoH-Sketch extracts features from either sampled or unsampled traffic. Next, by analyzing the extracted features, the classifier detects whether a DoH tunnel attack is occurring. If a DoH attack is detected, the tunneling tool classifier further analyzes the tunneling tool used in the attack.

V. EXPERIMENTS AND EVALUATION

We evaluated the detection performance, real-time capabilities, efficiency, and durability of TunnelEye and compared it with existing methods. This section presents the results of these experiments.

A. Experiment Setup

Testbed: For the real-time analysis experiment, we conducted experiments on an Ubuntu 20.04 virtual machine with four processor cores, a 100GB hard drive, and 8GB RAM. In other experiments, we performed experiments on a Windows 11 system equipped with an Intel[2] Core[3] i7-12700K CPU @ 3.61 GHz, a 4TB hard drive, and 128GB RAM.

Dataset: We validated the effectiveness of our proposed method using four datasets and two mixed datasets. The four individual datasets include two public datasets and two self-collected datasets,[2] as described in Table IV.

1) CIRA-CIC-DoHBrw-2020: The publicly available CIRA-CIC-DoHBrw-2020 dataset is provided by the Canadian Institute for Cybersecurity (CIC). The dataset employs a two-layered approach to capture non-DoH, benign DoH, and malicious DoH traffic. Non-DoH traffic is collected by accessing the top 10k Alexa websites through a web server. Benign and malicious DoH traffic is generated using browsers supporting DoH services and DNS tunneling attack tools. The dataset includes traffic samples generated by four major DoH servers (AdGuard, Cloudflare, Google DNS, and Quad) and three DNS tunneling tools (dns2tcp, DNSCat2, and iodine) that provide DoH services.

2) DoH-Extended: To ensure that our detection method can adapt to the latest traffic patterns and effectively identify various types of DoH tunneling traffic, we constructed the self-collected dataset DoH-Extended. This dataset was collected during multiple periods between July 2022 and November 2024.

The DoH-Extended dataset captures benign DoH traffic and non-DoH traffic using the same method as the publicly available dataset CIRA-CIC-DoHBrw-2020. For benign DoH traffic, a portion is sourced from two DoH servers (Quad and Google DNS) in CIRA-CIC-DoHBrw-2020, while the rest is collected from other mainstream DoH servers (AliDNS, DNSPod, 360, and OpenDNS) during our data collection. Regarding malicious DoH traffic, the public dataset CIRA-CIC-DoHBrw-2020 is limited in the coverage of attack tools, encompassing only three DNS tunneling tools, failing to represent the diversity of current DoH tunneling attacks comprehensively. To address this limitation, our self-collected dataset extends the coverage by incorporating three additional tunneling tools: DoHC2, dnstt, and DNSExfiltrator. It expands the applicability of the detection method, enabling it to identify and defend against a broader range of DoH tunneling attacks.

To collect attack traffic, we first need to construct the attack environment. Therefore, we begin by introducing the threat model, followed by the attack implementation and the traffic collection process.

Threat model. The prerequisite for establishing a DoH tunnel is that the attacker has already infected the target host through a Trojan or other means and deployed a malicious DoH tunneling tool. The attacker can send requests to a specified domain by manipulating the infected host. Since the attacker controls the domain and its resolution authority, they can design NS records to specify a controlled domain server to resolve the domain, thereby establishing a tunnel between the infected client and the controlled authoritative DNS server. The attacker can then use a Command and Control (C&C) server to transmit commands and execute attack actions through the tunnel.

Implementation of the Attack. The attacker registers a dedicated domain, "name.site," and configures an A record pointing to the IP address of the attacker-controlled server, where an authoritative DNS service is deployed. The attacker then creates two NS records, `send.name.site` and

[2]Registered trademark.
[3]Trademarked.
[4]These dataset are available upon request; please contact the corresponding author.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 22,2026 at 05:59:52 UTC from IEEE Xplore. Restrictions apply.

`receive.name.site`, both delegated to the authoritative server for `name.site`. From the attacker's perspective, `receive.name.site` is used to deliver control commands, while `send.name.site` is used to return (exfiltrate) sensitive data. The attack exhibits different behaviors in the waiting state and the active state.

When the DoH tunneling attack is in the waiting state, the malware deployed on the infected host periodically sends "waiting/heartbeat" status reports to the attacker. At fixed intervals, the infected host embeds the status information into subdomains of `receive.name.site` and issues DNS resolution requests. These requests are processed by DoH proxy/resolver providers in the network, and the embedded information is delivered to the attacker side.

When the attacker intends to steal sensitive information, control commands are embedded in DoH response messages and delivered from the attacker-controlled server to the infected host. Once the DoH tunneling attack starts, the malicious DoH tunneling client on the infected host pads and segments the victim's sensitive data, embeds the segments into subdomains of `send.name.site`, and continuously issues DNS resolution requests. After being resolved via the DoH proxy/resolver providers, the exfiltrated data are delivered to the attacker. Meanwhile, the attacker can also embed new control commands in response messages and deliver them back to the infected host.

Traffic Collection. For traffic capture, we deploy Wireshark on the infected host and record traffic during DoH tunneling attacks, storing traces in pcap format. We vary multiple attack parameters to collect traces across different scenarios, including different tunneling tools (DoHC2, dnstt, and DNSExfiltrator) and different DoH resolver providers (AliDNS, DNSPod, 360, and OpenDNS). During data collection, the infected host does not run any applications unrelated to the experiment; therefore, we label malicious DoH traffic by matching the infected host's IP address with the IP addresses of the selected DoH resolver providers, and treat the resulting flows as attack-generated malicious DoH traffic.

3) Trace A: To assess the impact of hash collisions on the DoH-Sketch in high-speed backbone networks, we used Trace A from the MAWI dataset [47]. Trace A was captured at sampling point F on the WIDE backbone network, covering 24 hours of traffic (from 23:45 on June 18 to 23:45 on June 19, 2024) on the transit link to the upstream ISP (Internet Service Provider). For privacy protection, the MAWI dataset anonymizes the data by removing payloads from TCP packets containing private user information and obfuscating source and destination addresses in the IP headers.

4) Trace B: To evaluate TunnelEye's resistance to interference in real networks, we collected Trace B and retained the payload. Trace B consists of 500 seconds of daily traffic (totaling 112GB) captured at the boundary of a campus network with a 10Gbps bandwidth. It includes a small amount of benign DoH traffic and various HTTPS traffic but does not contain malicious DoH traffic.

Mixed Datasets. The mixed datasets were constructed by combining subsets from above datasets. To simulate a network environment where different types of traffic coexist, we mix flows in a time-aligned manner. During this process, we define two roles during the mixing process: the *host flow*, which serves as the temporal container, and the *injected flow*, which is inserted into it. Each injected flow is assigned an insertion timestamp that determines when it begins within the host flow.

Let T_h denote the duration of the host flow, and T_j the duration of the injected flow. If $T_h > T_j$, the insertion timestamp is randomly chosen from the interval $(0, T_h - T_j)$, ensuring the injected flow fits entirely within the host flow. If $T_h \leq T_j$, the insertion timestamp is sampled from $(0, \lambda T_h)$, where the mixing coefficient λ is set to 0.05 in our experiments. This allows partial overlap when the host flow is relatively short. Finally, the timestamps of the injected flow are shifted by the chosen insertion time to align it within the host flow.

1) MixedSet1: By mixing CIRA-CIC-DoHBrw-2020 and DoH-Extended, we obtain MixedSet1. In an experiment comparing performance with similar work, MixedSet1 is used for 10-fold cross-validation. In another experiment evaluating interference resistance, 60% of MixedSet1 is used as the training set, and the remaining 40% is combined with Trance B to construct the test set.

2) MixedSet2: By mixing 40% of MixedSet1 with Trance B, we obtain MixedSet2. This dataset is used as the test set in an experiment comparing interference resistance with similar work.

Evaluation metrics: F1 score, precision, accuracy (ACC), and recall are the four most commonly used evaluation metrics for classification models. In malicious traffic detection tasks, false negative rate (FNR) and false positive rate (FPR) are critical factors affecting model performance. Therefore, to more comprehensively reflect TunnelEye's detection performance, we use F1 score, precision, accuracy, recall, FNR, and FPR as evaluation metrics.

B. Parameter Selection

This section discusses the selection of key parameters for features and DoH-Sketch.

1) ***Selecting Parameters ΔL for $L_{PL_scope_N}$ and $L_{TLS_scope_N}$*** To compute the features $L_{PL_scope_N}$ and $L_{TLS_scope_N}$, the parameter ΔL must first be determined. ΔL determines the length interval for recording the payload and TLS record length distributions, which is critical for capturing distributional differences between malicious DoH traffic and other types of traffic. An appropriately chosen ΔL can significantly enhance the accuracy and efficiency of malicious DoH traffic detection.

A small ΔL enables the model to capture fine-grained differences in payload and TLS record length distributions, which can improve detection accuracy. A large ΔL, on the other hand, reduces the number of intervals, which decreases computational and memory overhead during feature extraction and model training. Each setting offers a clear advantage: one enhances accuracy, the other improves efficiency. However, these benefits cannot be achieved simultaneously. This section aims to choose a ΔL that balances the two.

To maximize the distinction between two types of traffic distributions while maintaining acceptable resource usage, we employ Jensen-Shannon Divergence (JS Divergence) to

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 22,2026 at 05:59:52 UTC from IEEE Xplore. Restrictions apply.

TABLE V
D_{JS} OF MALICIOUS AND BENIGN TRAFFIC ACROSS DIFFERENT FEATURES

ΔL	$D_{JS}(L_{PL_sope_N})$	$D_{JS}(L_{TLS_sope_N})$
50	0.647	0.548
100	0.643	0.542
200	0.640	0.537
400	0.639	0.531

quantify their distributional differences and determine an appropriate ΔL. JS Divergence is a method for measuring the similarity between two probability distributions, where a JS Divergence closer to 1 indicates a more significant difference between the two distributions. Although the payload length distribution and TLS record length distribution are frequency distributions, they can approximate probability distributions when the sample size is sufficiently large. JS Divergence is defined as:

$$D_{\mathrm{JS}}\left(P \parallel Q\right) = \frac{1}{2} D_{\mathrm{KL}}\left(P \parallel M\right) + \frac{1}{2} D_{\mathrm{KL}}\left(Q \parallel M\right) \quad (1)$$

In Formula (1), P and Q represent the distributions of malicious DoH traffic and other traffic, respectively. $M = \frac{1}{2}(P+Q)$ is their average distribution, and D_{KL} refers to the Kullback-Leibler Divergence, which is calculated as shown in Formula (2). Here, p_i and q_i represent the frequency of the i-th length interval in distributions P and Q, respectively.

$$\begin{cases} D_{\mathrm{KL}}(P\|M) = \sum_{i=1}^{n} p_i \log\left(\frac{m_i}{p_i}\right) \\ D_{\mathrm{KL}}(Q\|M) = \sum_{i=1}^{n} q_i \log\left(\frac{m_i}{q_i}\right) \end{cases}, m_i = \frac{p_i + q_i}{2} \quad (2)$$

We calculate the D_{JS} values corresponding to different ΔL, as shown in Table V. The D_{JS} values between malicious DoH traffic and other traffic over both the $L_{PL}_scope_N$ and $L_{TLS}_scope_N$ are all greater than 0.5, indicating a clear distributional difference between the two types of traffic. The maximum D_{JS} is achieved when $\Delta L = 50$, which provides the best separation between malicious and benign traffic. As ΔL increases from 50 to 400, the D_{JS} values decrease only slightly, by 1.2% for the $L_{PL}_scope_N$ feature and 3.1% for the $L_{TLS}_scope_N$ feature. This indicates that the method is not highly sensitive to the choice of ΔL. Therefore, although $\Delta L = 50$ yields the highest D_{JS}, we adopt $\Delta L = 200$ in practice, as it substantially reduces the feature extraction overhead while preserving nearly the same discriminative power.

*2) **Selecting the Parameters d and w for DoH-Sketch*** The parameters d and w of DoH-Sketch are closely related to memory occupancy and hash collision rate. Appropriate configuration of d and w enables DoH-Sketch to handle large-scale traffic with low hash collision rates while utilizing limited memory. Therefore, it is necessary to select a set of d and w that balances memory usage and hash collisions. The memory usage of DoH-Sketch is mainly determined by its data structure, which consists of $d \times w$ buckets, where d and w are the number of rows and columns, respectively. Each bucket uses s bytes, so the total memory usage is $d \times w \times s$. Since s is fixed, memory usage depends on d and w.

TABLE VI
IMPACT OF d AND w ON HASH COLLISION

Key Count	d	w	Full Collision Rate (%)	Partial Collision Key Count	Partial Collision Rate (%)
1M	3	2^{24} (∼16M)	0.0000%	218	0.0218%
		2^{25} (∼32M)	0.0000%	31	0.0031%
		2^{26} (∼64M)	0.0000%	4	0.0004%
		2^{27} (∼128M)	0.0000%	1	0.0001%
	4	2^{24} (∼16M)	0.0000%	7	0.0007%
		2^{25} (∼32M)	0.0000%	1	0.0001%
		2^{26} (∼64M)	0.0000%	0	0.0000%
		2^{27} (∼128M)	0.0000%	0	0.0000%
3M	3	2^{24} (∼16M)	0.0000%	13141	0.4380%
		2^{25} (∼32M)	0.0000%	1904	0.0635%
		2^{26} (∼64M)	0.0000%	291	0.0097%
		2^{27} (∼128M)	0.0000%	43	0.0014%
	4	2^{24} (∼16M)	0.0000%	2126	0.0709%
		2^{25} (∼32M)	0.0000%	163	0.0054%
		2^{26} (∼64M)	0.0000%	16	0.0005%
		2^{27} (∼128M)	0.0000%	0	0.0000%
5M	3	2^{24} (∼16M)	0.0000%	85370	1.7074%
		2^{25} (∼32M)	0.0000%	13419	0.2684%
		2^{26} (∼64M)	0.0000%	1873	0.0375%
		2^{27} (∼128M)	0.0000%	257	0.0051%
	4	2^{24} (∼16M)	0.0000%	21849	0.4370%
		2^{25} (∼32M)	0.0000%	1873	0.0375%
		2^{26} (∼64M)	0.0000%	133	0.0027%
		2^{27} (∼128M)	0.0000%	4	0.0001%

Hash collisions can cause overestimation, thereby degrading estimation accuracy, which makes collision analysis critical for sketch-based methods. We consider two collision types. A full collision occurs when two packet keys A and B map to the same counter in every row, leading to an overestimate. A partial collision occurs when A and B collide in only a subset of rows. Although partial collisions are less severe, they can still introduce non-negligible error when they occur frequently. We evaluate full and partial collision rates using one million, three million, and five million unique packet keys under different settings of d (rows) and w (columns). The results are reported in Table VI.

The results indicate that collision rates decrease with increasing d and w. When $d = 4$ and the number of keys is close to w/10, the partial collision rate is under 0.01%. Among one million packet keys, only a few hundred or fewer keys collide. Moreover, a partial collision does not necessarily result in an estimation error. As a result, the probability of actual errors can be even lower, which supports reliable feature extraction.

Therefore, we set $d = 4$ and $w = 2^{\lceil \log_2(10F) \rceil}$ in the subsequent experiments, where F denotes the number of network flows. Under this configuration, reliable feature extraction at the million-flow scale requires approximately 1.25 GB of memory, which is affordable for most computers.

C. Feature Predictive Power and Computational Cost Comparison

We compare our proposed unidirectional-flow feature set with a baseline feature set that has been widely adopted in

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 22,2026 at 05:59:52 UTC from IEEE Xplore. Restrictions apply.

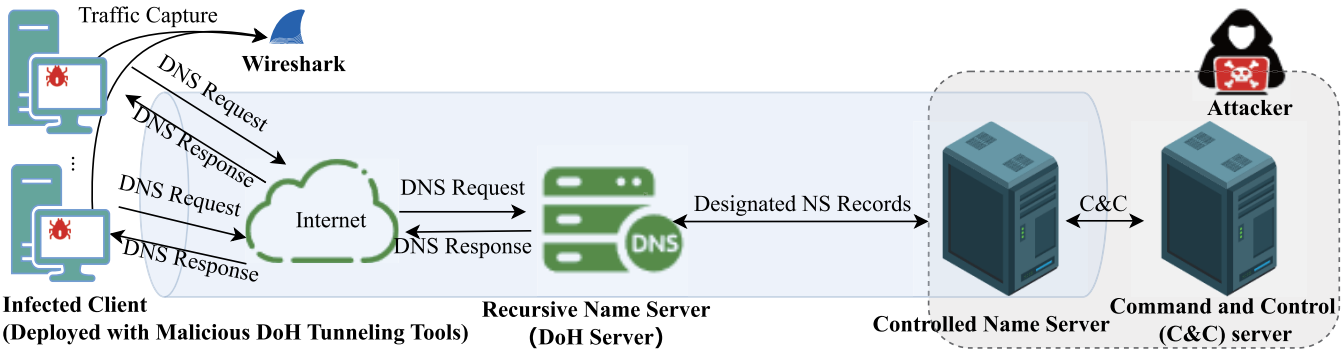


Fig. 7. Topology of malicious DoH tunneling traffic generation and collection.

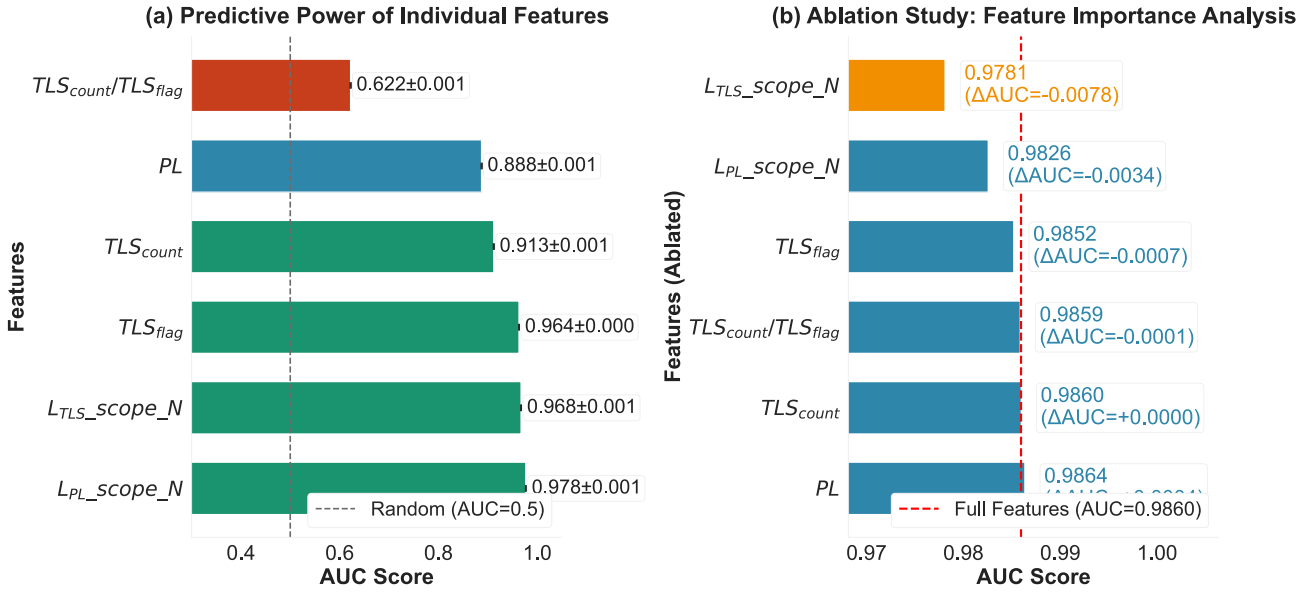


Fig. 8. Predictive power and importance of our features.

prior studies [15], [41], [44]. We use only unidirectional traffic throughout this section. All experiments in this section are conducted on the public CIRA-CIC-DoHBrw-2020 dataset.

1) Feature Predictive Power Analysis and Ablation Study: To evaluate the predictive power and relative importance of the features in our unidirectional-flow feature set, we assess all features using a logistic regression model after standardization and report AUC (area under the ROC curve) as the evaluation metric. The experiments consist of three stages: i) evaluating the classification performance when each feature is used independently; ii) training the model with the complete feature set to obtain an overall baseline; and iii) performing an ablation study by removing one feature at a time and quantifying its importance by the resulting AUC drop, where a larger decrease indicates a greater contribution. The results are shown in Fig. 8 and Fig. 9.

The individual feature evaluation indicates that our proposed feature set has stronger overall discriminative power. Most features achieve an AUC above 0.9 when used individually, clearly outperforming the baseline features on average. In particular, L_{PL}_scope_N and L_{TLS}_scope_N perform best, suggesting that the length-distribution patterns of malicious DoH traffic differ substantially from those of benign traffic.

The ablation experiments further confirm the critical contribution of our features to the overall performance. Removing most features leads to a decrease in AUC, indicating that the majority of features contribute positively. Among them, L_{PL}_scope_N and L_{TLS}_scope_N have the largest impact, consistent with the individual feature results. Notably, removing *PL* slightly increases the AUC by only 0.0004, suggesting that *PL* is partially redundant and that its information is likely captured by the finer-grained distribution feature L_{PL}_scope_N. Nevertheless, we retain *PL* because it is computationally inexpensive and highly interpretable, and because different models may exploit features differently; retaining it can improve robustness under non-ideal conditions. In contrast, the baseline feature set shows clear over-reliance on a single feature, as removing *PacketLengthMedian* causes a substantial performance drop.

2) Feature Computation Cost: To evaluate the computational cost of the feature set proposed in this paper against the baseline feature set, we conduct a quantitative analysis of their

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 22,2026 at 05:59:52 UTC from IEEE Xplore. Restrictions apply.

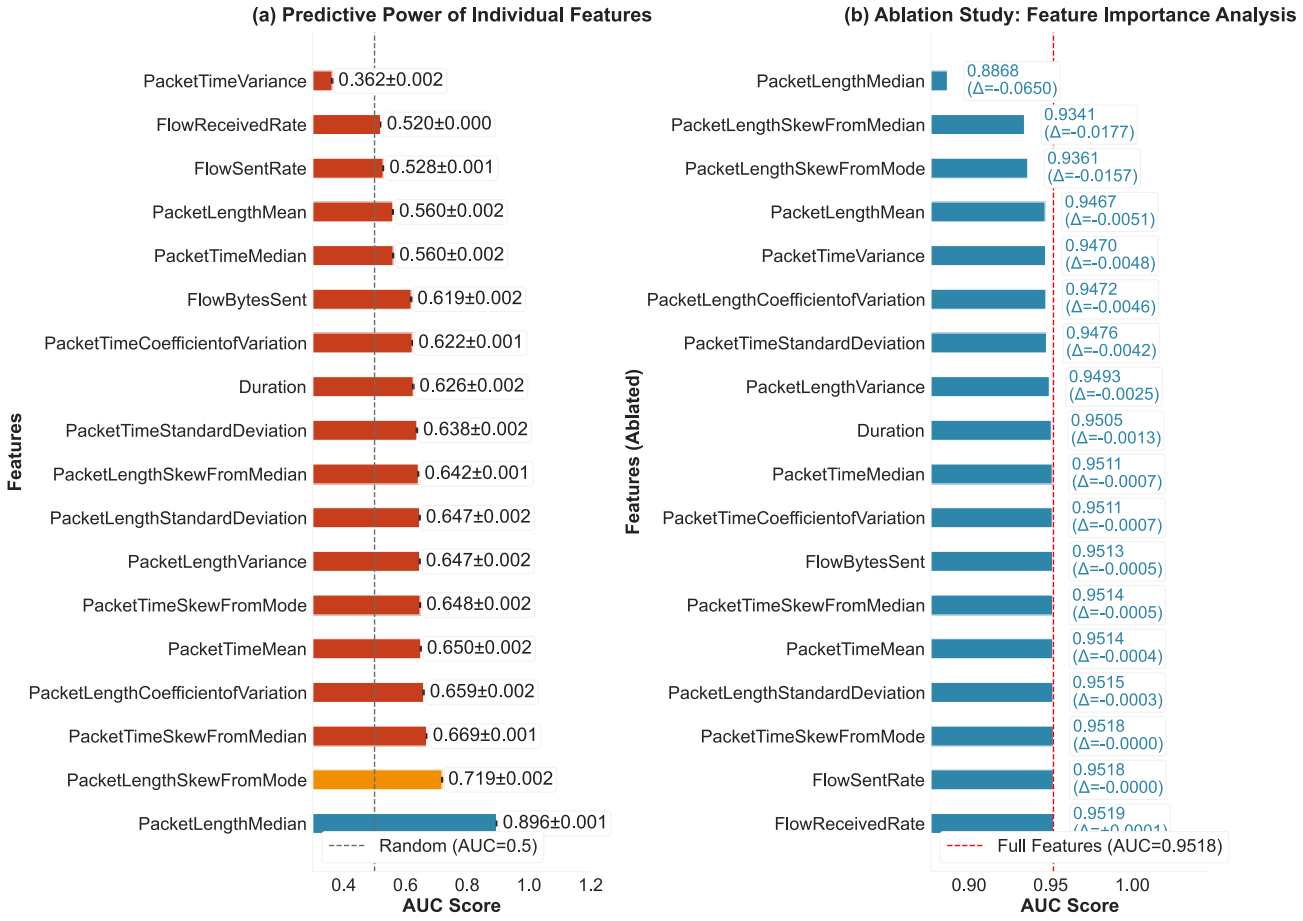


Fig. 9. Predictive power and importance of baseline features.

resource consumption and scalability under different traffic scales. The test data are selected from the public CIRA-CIC-DoHBrw-2020 dataset. Each pcap file is truncated by packet count into five scale levels, namely 100k, 200k, 300k, 400k, and 500k packets, with the corresponding file size increasing approximately linearly from 35.6 MB to 142.2 MB.

All experiments are performed under identical hardware and operating system conditions. For each pcap file, two feature-extraction tools (corresponding to the two feature sets) are executed in 10 independent runs, and the reported results are averaged to mitigate run-to-run randomness. During each run, we record wall-clock execution time, average CPU utilization, peak CPU utilization, average Resident Set Size (RSS) memory usage, and peak RSS memory usage. All metrics are automatically collected using non-intrusive system resource monitoring tools to avoid interfering with the programs under test.

As summarized in Table VII, our feature set is consistently more computationally efficient and exhibits better scalability as traffic volume increases. Across all scales, the processing time of our extractor remains stable at about 4.36–4.45 s, whereas the baseline requires 108.8–293.9 s, making our approach approximately 24 to 66 times faster. Memory usage shows an even larger scalability gap: our peak RSS stays near 1.04 GB across scales, while the baseline peak RSS grows from 1.89 GB to 7.93 GB as traffic increases. Overall, as the traffic scale increases, the baseline feature set incurs heavy and continually growing computational overhead, whereas the resource consumption of our feature set remains nearly unchanged. This demonstrates strong scalability and makes our feature set well suited for large-scale traffic processing.

D. Detection Performance

This section evaluates the performance of various artificial intelligence models in detecting malicious DoH traffic and selects the optimal model. Additionally, we assess TunnelEye's capability to identify malicious DoH attack sources, verifying its applicability and accuracy across different tunneling tools.

1) Performance Comparison of Machine Learning Models for Malicious DoH Detection: To identify the most suitable detection model for malicious DoH in high-speed backbone networks, we compared the performance of various classification models on the CIRA-CIC-DoHBrw-2020 and DoH-Extended datasets. Unlike deep learning models, machine learning models benefit from domain knowledge to optimize feature engineering and offer significant advantages in computational resource consumption and explainability. Therefore, we select three machine learning models with

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 22,2026 at 05:59:52 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
COMPUTATIONAL COST

Packet numbers	Size (MB)	Feature set	Processing Time (s)	CPU usage (average)	CPU usage (peak)	RSS memory (average)	RSS memory (peak)
100k	35.6	Our Set	4.4518	2.32%	45.96%	870.70 MB	1044.45 MB
		Baseline Set	108.8133	0.21%	100.30%	1327.64 MB	1892.45 MB
200k	59.7	Our Set	4.3579	0.21%	4.26%	864.08 MB	1044.72 MB
		Baseline Set	207.0457	0.05%	41.80%	2277.83 MB	3286.66 MB
300k	90.9	Our Set	4.4126	0.21%	4.26%	886.49 MB	1044.22 MB
		Baseline Set	221.1762	0.03%	41.90%	3185.30 MB	4772.88 MB
400k	118.3	Our Set	4.36	1.26%	23.88%	897.28 MB	1044.09 MB
		Baseline Set	293.8799	0.03%	41.98%	4382.72 MB	6295.17 MB
500k	142.2	Our Set	4.3902	1.24%	24.04%	883.44 MB	1044.31 MB
		Baseline Set	289.2166	0.03%	40.38%	5215.07 MB	7928.74 MB

TABLE VIII
COMPARISON OF DETECTION PERFORMANCE AMONG DIFFERENT MODELS

Dataset	Algorithm	Direction	Metrics (%) 95% CI					
			ACC	Precision	Recall	F1	FNR	FPR
CIRA-CIC-DoHBrw-2020	RF	fwd	98.02 [97.96, 98.07]	99.92 [99.90, 99.93]	96.43 [96.32 ,96.53]	98.14 [98.09, 98.20]	3.57 [3.47, 3.68]	0.07 [0.06, 0.08]
		bwd	98.48 [98.41, 98.55]	99.76 [99.73,99.79]	98.30 [98.21,98.38]	99.02 [98.98, 99.07]	1.70 [1.62, 1.79]	0.71 [0.62, 0.82]
	XGB	fwd	99.88 [99.86, 99.90]	99.88 [99.86, 99.90]	99.88 [99.86, 99.90]	99.88 [99.87, 99.89]	0.12 [0.10, 0.14]	0.11 [0.09, 0.13]
		bwd	99.70 [99.67, 99.75]	99.79 [99.76, 99.82]	99.84 [99.81, 99.87]	99.82 [99.80, 99.84]	0.16 [0.13, 0.19]	0.63 [0.54, 0.73]
	LightGBM	fwd	99.88 [99.86, 99.89]	99.88 [99.86, 99.90]	99.89 [99.87, 99.91]	99.88 [99.87, 99.90]	0.11 [0.09, 0.13]	0.11 [0.09, 0.13]
		bwd	99.72 [99.69, 99.75]	99.80 [99.77, 99.83]	99.86 [99.83, 99.88]	99.83 [99.81, 99.85]	0.14 [0.12, 0.17]	0.61 [0.52, 0.70]
DoH-Extended	RF	fwd	99.56 [99.53, 99.59]	99.94 [99.92, 99.95]	99.80 [99.77, 99.83]	99.87 [99.85, 99.88]	0.20 [0.17,0.23]	0.08 [0.06, 0.10]
		bwd	98.97 [98.88, 99.05]	99.99 [99.99, 1.00]	98.98 [98.89, 99.07]	99.49 [99.44, 99.53]	1.02 [0.93, 1.11]	0.02 [0.00,0.14]
	XGB	fwd	99.96 [99.95, 99.97]	99.98 [99.97, 99.99]	99.98 [99.97, 99.99]	99.98 [99.98, 99.99]	0.01 [0.01, 0.02]	0.03 [0.02, 0.04]
		bwd	99.96 [99.95, 99.98]	99.99 [99.98, 1.00]	99.99 [99.99, 1.00]	99.99 [99.99, 1.00]	0.00 [0.00, 0.01]	0.12 [0.05, 0.29]
	LightGBM	fwd	99.96 [99.95, 99.97]	99.98 [99.97, 99.99]	99.99 [99.98, 99.99]	99.98 [99.98, 99.99]	0.01 [0.01,0.02]	0.02 [0.01,0.04]
		bwd	99.96 [99.94, 99.98]	99.99 [99.97, 99.99]	99.99 [99.99, 1.00]	99.99 [99.99, 1.00]	0.00 [0.00, 0.01]	0.15 [0.07, 0.32]

superior performance: Random Forest (RF), XGBoost (XGB), and LightGBM for performance evaluation.

To ensure comparability and avoid overfitting, the three models are trained under the same complexity constraints and use regularization, subsampling, and early stopping to control model complexity. Random Forest, XGB, and LightGBM cap the maximum tree depth at 6 and use row and feature subsampling to reduce variance. Random Forest additionally uses bootstrap sampling and enforces minimum sample requirements for splits and leaves. XGB and LightGBM include L2 regularization and share the same upper bound on the number of trees; within each training fold, an 8:2 split is used to form an internal validation set for early stopping, and training halts after 50 consecutive rounds without improvement in the validation loss. Generalization is assessed with stratified 10-fold cross-validation using out-of-fold predictions, where the internal validation set is drawn only from training folds and the test fold is never used for training, early stopping, or hyperparameter tuning, which prevents test-set leakage.

For evaluation, we quantify uncertainty and test statistical significance using the out-of-fold predictions. We quantify uncertainty and assess statistical significance by constructing 95% confidence intervals for accuracy and F1 via bootstrap resampling. Wilson score intervals are reported for class-wise Precision, Recall, false negative rate (FNR), and false positive rate (FPR). Differences between models are evaluated using McNemar's test on paired out-of-fold predictions. The results of ten-fold cross-validation are shown in Table VIII.

As shown in Table VIII, the ACC, Precision, Recall, and F1 scores obtained by XGB and LightGBM for identifying malicious DoH traffic across various monitoring directions all exceed 99%, indicating that our features provide strong

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 22,2026 at 05:59:52 UTC from IEEE Xplore. Restrictions apply.

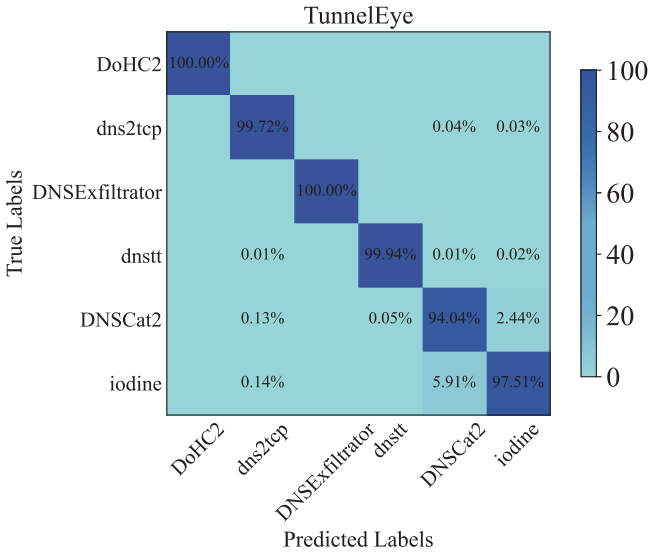


Fig. 10. Confusion matrix for attack source identification.

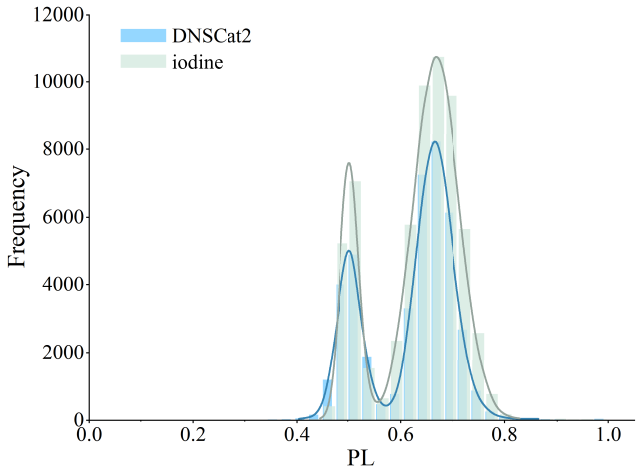


Fig. 11. Comparison of the frequency distributions of PL in DoH instances generated by DNSCat2 and iodine.

representational capability. Among the three algorithms, the performance of XGB and LightGBM is highly comparable. McNemar's tests across different datasets and monitoring directions yield p-values greater than 0.05, failing to show a statistically significant difference in detection performance between the two models. Given LightGBM's advantages in training efficiency, memory usage, and scalability for large datasets, together with its support for parallel computing and practical imbalance-aware training (e.g., weighting), we consider LightGBM more suitable for long-term deployment in backbone networks. Therefore, all subsequent experiments use the TunnelEye detection framework based on the LightGBM model.

2) Evaluation of Attack Source Identification Accuracy: We evaluate TunnelEye's performance in identifying the sources of malicious DoH traffic. The public CIRA-CIC-DoHBrw-2020 dataset and our self-collected DoH-Extended dataset contain a total of six types of malicious DoH tunneling tools: dns2tcp, DNSCat2, iodine, dnstt, DNSExfiltrator, and DoHC2. We assess TunnelEye's accuracy in detecting these six attack sources using ten-fold cross-validation.

As shown in the confusion matrix in Fig. 10, the identification accuracy for the six tools is generally high, with four tools achieving accuracy above 99.7%. However, the distinction ability for DNSCat2 and iodine is slightly lower, recognition accuracies of 94.04% and 97.51%, respectively. Through further analysis of the most influential feature in TunnelEye's dependency, PL, which refers to the ratio of packets with payloads to the total number of packets, we observed that the frequency distribution of PL for DNSCat2 and iodine (refer to Fig. 11) exhibits a high degree of overlap, which means that the ratio of packets with payloads in the malicious DoH traffic generated by DNSCat2 and iodine is very similar, resulting in reduced performance when distinguishing between these tools.

E. Model Sensitivity to the Threshold θ

In DoH-Sketch, the feature extraction threshold θ indicates the packet count required to trigger feature extraction. The choice of θ determines the trade-off between detection speed and accuracy. If θ is set too large, more packets must be accumulated before extraction is triggered, increasing the feature output delay and thereby slowing detection and alerting. Conversely, if θ is too small, features can be generated more quickly, but because the number of observed packets in a bucket is insufficient, statistics become more susceptible to random fluctuations and transient noise, leading to unstable features and degraded detection accuracy. Therefore, it is necessary to conduct sensitivity experiments to systematically evaluate the model's dependence on θ, identify stable and fragile performance regions as θ varies, and provide guidance for selecting θ.

TABLE IX
FPR AND FNR FOR VARIOUS θ

θ	FPR	FNR
10	1.44%	21.90%
50	0.40%	0.96%
100	0.19%	0.18%
150	0.16%	0.11%
200	0.12%	0.11%

We first set θ to 100 to extract features from the public CIRA-CIC-DoHBrw-2020 dataset, and then trained and tested the model based on these features. We then applied the trained model to the same test set, re-extracting features and constructing the corresponding test inputs using $\theta \in \{10, 50, 150, 200\}$. We evaluated the false positive rate (FPR) and false negative rate (FNR) under various θ to assess the model's sensitivity to θ.

Table IX shows that the model is highly sensitive to θ in the range of 10–50: increasing θ to 50 reduces the FNR by approximately 21%. When θ increases from 50 to 100, the FNR decreases further by only 0.78%; the model remains sensitive to θ, but to a substantially lesser degree. In the range of 100–200, both FPR and FNR become nearly stable, indicating that the model is no longer sensitive to changes in θ. Therefore, a practical strategy is to set $\theta = 200$ during

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 22,2026 at 05:59:52 UTC from IEEE Xplore. Restrictions apply.

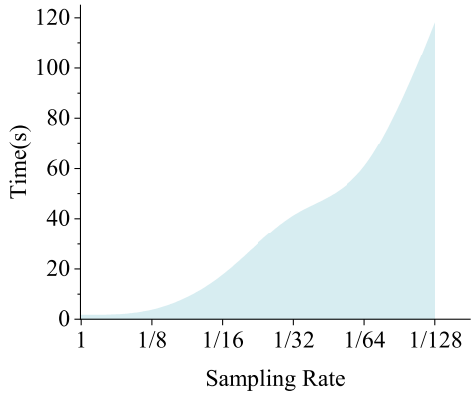


Fig. 12. Attack detection time.

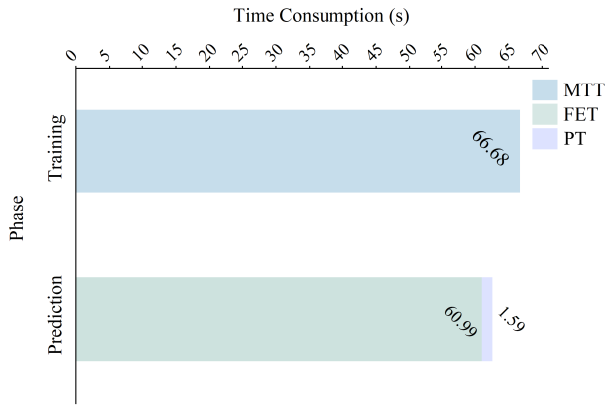


Fig. 13. Time consumption of TunnelEye during training and prediction phases.

training to learn from more stable features, and $\theta = 100$ during deployment to achieve faster alerting with only a small increase in FPR.

F. Real-Time Performance

To evaluate the real-time performance of TunnelEye, we measure the time interval from the initiation of an attack to its detection. We conduct malicious DoH tunneling attacks using dnstt and deploy TunnelEye between the victim client and the attacker-controlled DoH server for real-time detection. TunnelEye monitors bidirectional traffic but identifies malicious DoH traffic based solely on unidirectional traffic. By performing fifty attack experiments at each sampling rate, we determine the average attack detection time of TunnelEye under different sampling rates, as shown in Fig. 12.

As shown in Fig. 12, without sampling, the average attack detection time is only 2.05 seconds. The average detection time gradually increases as the sampling rate decreases because features are extracted when the accumulated number of packets reaches a threshold. It takes longer to reach the threshold at a lower sampling rate. However, even at the sampling rate of 1/128, the average detection time does not exceed 2 minutes, demonstrating that TunnelEye offers excellent real-time performance and is suitable for deployment in high-speed backbone networks.

G. Detection Efficiency Analysis

During the experiments, several key performance metrics are measured to evaluate time efficiency. We record the model training time (MTT) to assess the time consumed during the training process. Using feature extraction time (FET) and model prediction time (PT) to quantify the time required during the prediction phase. The experimental results are shown in Fig. 13, where both the training and testing phases use the publicly available CIRA-CIC-DoHBrw-2020 dataset.

Fig. 13 demonstrates the high time efficiency of TunnelEye. Its MTT and PT are 66.68 seconds and 1.59 seconds, respectively, with the average time to predict a traffic sample being only 6 microseconds. This high efficiency is because the LightGBM algorithm discretizes continuous feature values into specific bins (i.e., histogram buckets), thereby reducing the computational load during node splitting and significantly enhancing processing speed while minimizing memory usage. Additionally, the FET for the 67.5GB public dataset is 60.99 seconds, corresponding to a processing speed of 8.86 Gbps. This performance is attributed to our DoH-Sketch algorithm, which does not require sorting or performing multiple complex statistical operations during feature extraction, significantly reducing memory requirements and improving overall performance.

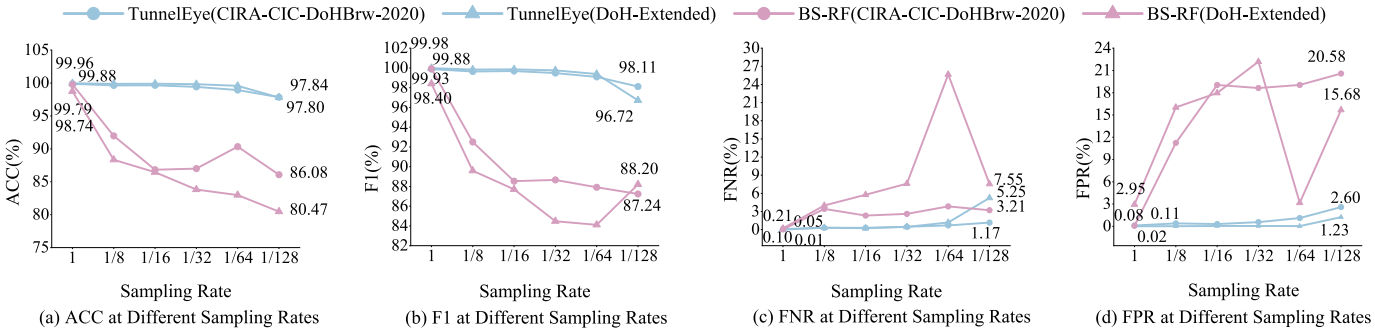
We further evaluate the efficiency of DoH-Sketch in real-world networks. Performance testing on Trace B shows a processing rate of 35.17 Gbps. Under the sampling scenario with $R_{\text{sample}} = 1/8$, the processing speed of DoH-Sketch increases to 113.59 Gbps, making it suitable for deployment in high-speed backbone networks with a bandwidth of 100 Gbps. DoH-Sketch processes Trace B faster than public datasets because non-HTTPS traffic in real networks is excluded from processing, whereas public datasets consist almost entirely of HTTPS traffic. These results indicate that DoH-Sketch can meet the requirements for applications in backbone networks with 100 Gbps bandwidth.

H. Hash Collision Analysis

To analyze the effect of hash collisions on feature accuracy, we conducted the experiments. The hash function maps high-dimensional data to a low-dimensional storage structure in the Sketch algorithm. Since the range of hash values is limited while the input data can be massive, different data items may be mapped to the same location, resulting in hash collisions. These collisions cause originally independent data to mix, leading to frequency estimation errors and reducing feature accuracy.

In DoH-Sketch, hash collisions occur when packets with different *Keys* are mapped to the same bucket. A feature conflict occurs when all d buckets corresponding to a *Key* experience hash collisions. Feature conflicts are closely related to the count of long flows (Long Flow Count, LFC) in the network, as long flows, which contain more packets than short flows, have a more significant impact on feature conflicts. Furthermore, feature conflicts are also closely associated with the number of buckets. DoH-Sketch consists of a two-dimensional

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 22,2026 at 05:59:52 UTC from IEEE Xplore. Restrictions apply.



(a) ACC at Different Sampling Rates (b) F1 at Different Sampling Rates (c) FNR at Different Sampling Rates (d) FPR at Different Sampling Rates

Fig. 14. Detection accuracy comparison at different sampling rates.

TABLE X
P_{FC} UNDER DIFFERENT R_{sample}

R_{sample}	*LFC*	Theoretical P_{FC}	Measured P_{FC}
1/8	26,160,146	0.000076	0.000000
1/16	12,775,364	5.04×10^{-6}	0.000000
1/32	10,141,629	1.85×10^{-6}	0.000000
1/64	4,687,917	9.54×10^{-7}	0.000000
1/128	2,917,989	1.34×10^{-7}	0.000000

TABLE XI
INTERFERENCE RESISTANCE COMPARISON

Method	Environment	FNR(%)	FPR(%)
TunnelEye	No Background Traffic	0.08	0.10
	With Background Traffic	0.09	0.67
BS-RF [15]	No Background Traffic	5.42	4.43
	With Background Traffic	5.63	31.79

array of buckets with d rows and w columns ($d = 4, w = 2^{28}$). Based on these factors, the calculation of P_{FC} is as follows:

$$P_{FC} = \left(1 - \left(1 - \frac{1}{w}\right)^{LFC-1}\right)^{d} \tag{3}$$

We evaluate the feature conflict risk of DoH-Sketch on Trace A by measuring the LFC and P_{FC}, and computing the theoretical P_{FC} according to Equation (3). We analyze these results, summarized in Table X, to assess the level of feature conflict observed and how well it aligns with theoretical expectations.

Table X shows that when R_{sample} is 1/8, DoH-Sketch can continuously monitor backbone traffic for 24 hours with a P_{FC} of only 0.000076. In our experiments, the measured P_{FC} is 0.000000. Since the theoretical analysis also predicts an extremely small P_{FC}, the experimental results validate the theoretical estimate. Moreover, the measured P_{FC} is lower than the theoretical value, likely because multiple flows may share the same IP address, resulting in fewer distinct keys in practice and therefore a lower collision probability.

I. Comparison With Similar Work

We compare our work with the research by Zebin et al. [15], which proposes a Balanced Stacked Random Forest (BS-RF) model. The BS-RF model achieves accuracy, recall, and F1 scores exceeding 99.9% on the public dataset CIRA-CIC-DoHBrw-2020, making it one of the most representative works in malicious DoH detection.

The BS-RF model [15] extracts 29 features from the CIRA-CIC-DoHBrw-2020 dataset, grouped into five categories: flow duration, flow byte count, packet length, packet time, and packet inter-arrival delay. Among these features, the packet length mode is highly contributive but flawed: the mode is 68 bytes for 88.7% of malicious DoH traffic and 66 bytes for 50.5% of benign DoH traffic. These values correspond to the typical size of TCP ACK packets, with the two-byte difference resulting from the different operating systems used during data collection. Specifically, the 68-byte ACK packets were captured on a Linux platform, while the 66-byte ACK packets were captured on a Windows platform. This discrepancy indicates that the packet length mode is influenced by environmental factors rather than intrinsic characteristics of malicious DoH traffic. Consequently, we excluded this feature from all comparison experiments.

1) Performance Comparison in Backbone Network Scenarios: We compare the detection performance of the proposed TunnelEye with the BS-RF model [15]. In this experiment, we simulate a scenario where sampling is performed on a backbone network with widely deployed asymmetric routing. We split the public CIRA-CIC-DoHBrw-2020 dataset and our DoH-Extended dataset into unidirectional traffic and set five sampling rates (1/8, 1/16, 1/32, 1/64, and 1/128). The detailed evaluation results using ten-fold cross-validation are shown in Fig. 14.

As shown in Fig. 14, as the sampling rate decreases, the ACC and F1 scores generally decrease, while the FNR and FPR scores increase. However, TunnelEye is far less affected by sampling interference than the BS-RF model [15]. TunnelEye outperforms BS-RF [15] at all five sampling rates, with a FPR not exceeding 3%. The significant performance decline of BS-RF [15] after sampling is due to the combined effects of sampling and unidirectional traffic. Specifically, eight of the features used by BS-RF [15] are missing under the unidirectional traffic scenario. Additionally, sampling interferes with features based on packet time intervals. If key packets are discarded during sampling, it also affects features based on flow byte counts.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 22,2026 at 05:59:52 UTC from IEEE Xplore. Restrictions apply.

2) Comparison of Resistance to Interference in Real-World Networks: We compare the interference resistance of TunnelEye and BS-RF [15] in detecting malicious DoH traffic in a real network environment, using false positive and false negative rates as metrics. The Non-DoH traffic in the CIRA-CIC-DoHBrw-2020 and DoH-Extended datasets is HTTPS traffic generated by simulated web access. While web access is one of the most common sources of HTTPS traffic, it does not comprehensively reflect the diversity of HTTPS patterns observed in real-world networks. To better simulate realistic conditions, we captured Trace B from the campus network boundary as a background traffic dataset. Trace B includes various HTTPS traffic patterns, such as web access, streaming media transmission, and other HTTPS-based application communications. In the experiment, we use 60% of MixedSet1 for training and MixedSet2 for testing, where MixedSet2 is constructed by combining the remaining 40% of MixedSet1 with Trance B. The experimental results are presented in Table Table XI.

As shown in Table XI, under the interference of 20,000 daily background traffic instances, TunnelEye still maintains relatively low FNR and FPR. In contrast, the BS-RF [15] model exhibits significant performance degradation due to background traffic interference, with the FPR increasing by 27.36%. The performance drop is attributed to BS-RF [15] focusing primarily on superficial features, such as packet length and packet timing, which do not capture the core characteristics of the DoH protocol. As a result, BS-RF [15] achieves a relatively high accuracy in classifying malicious DoH traffic but is easily affected by other HTTPS traffic, misclassifying non-DoH traffic as malicious DoH. By analyzing the decision paths of misclassified samples in the BS-RF [15] model, we find that the top three features most responsible for misclassifying non-DoH traffic as malicious DoH are the median packet time, the mean packet time, and the packet time skew, which indicates that HTTPS traffic with regular time intervals in daily traffic shares some similarities in packet time distribution with malicious DoH traffic. In summary, when facing the complex environment of diverse HTTPS traffic in real network scenarios, TunnelEye demonstrates better robustness than the BS-RF [15] model.

VI. Explaining Model Decisions

In this section, we employ SHAP (SHapley Additive ExPlanations) to investigate the model's decision-making process. Based on Shapley values from game theory, it quantifies each feature's marginal contribution to predictions, revealing the direction and magnitude of their impact. This analysis helps identify key predictive features and uncovers complex feature-prediction relationships, enhancing model transparency and interpretability.

A. Feature Importance Plots

In this section, we use SHAP values to analyze feature importance. After training the model under the same experimental setting as the main evaluation, we compute SHAP values on the test set to quantify each feature's contribution to the predictions. SHAP values indicate how much each feature contributes to a given prediction.We then rank features by the mean absolute SHAP value mean (|SHAP|) and plot the feature importance for all features.

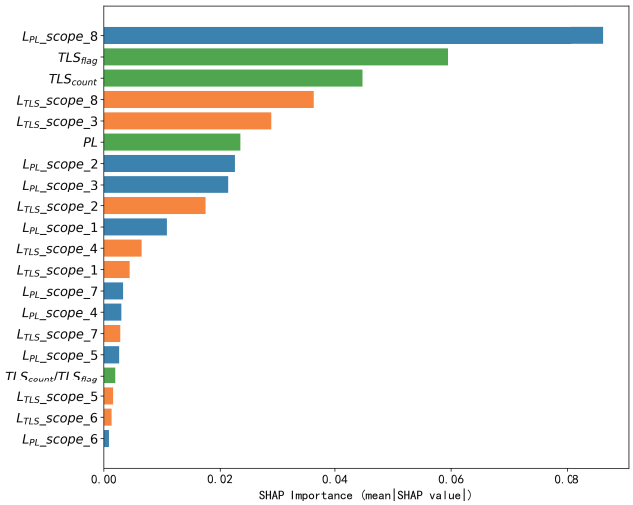


Fig. 15. SHAP summary plot.

As shown in Fig. 15, the SHAP values indicate that $L_{PL}_scope_8$ has the largest contribution to the model, far exceeding the other features, which suggests it is the key factor driving the predictions. TLS_{flag} and TLS_{count} also contribute substantially, partly because they help differentiate non-DoH traffic from DoH traffic. Some ranges of the length-distribution features show smaller contributions, likely because network traffic sizes vary widely; however, overall, the length-distribution features still play an important role in the model.

B. Explaining a Malicious and a Non-DoH Test Sample

To transparently demonstrate the model's decision-making process, we visualize the contributions of individual features to the prediction using contribution tables and plots. Fig. 16 and Fig. 17 present decision explanations for a malicious DoH and a Non-DoH sample, respectively.

Taking the correctly predicted malicious DoH instance in Fig. 16 as an example, the contribution plot on the right breaks down the positive or negative roles each feature plays in the current decision, while the contribution table on the left lists specific feature values (for clarity, we display only the top 10 most impactful features). Compared to typical malicious traffic, the $TLS_{flag} = 1$ of this sample indicates that all its packets contain TLS record headers at the payload beginning—a prominent DoH characteristic that positively (+ 24.47%) contributes to the model's classification as "malicious." Furthermore, the TLS Record Length distribution features provide a high cumulative positive contribution (+ 32.61%). However, this sample uniquely has no packets in the third payload length distribution interval, which differs from the norm for malicious samples and slightly reduces the predicted probability of being malicious (−0.21%).

Fig. 17 illustrates a similar analysis for a Non-DoH test sample. Notably, for this sample, both TLS_{flag} and TLS_{count} equal 0.04, whereas in Fig. 16, TLS_{flag} is higher. These two extremely low feature values (+ 30.13%) play a decisive role in the model correctly predicting the traffic as Non-DoH, whereas

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 22,2026 at 05:59:52 UTC from IEEE Xplore. Restrictions apply.

Feature	Effect
$TLS_{flag} = 1$	+24.47%
$L_{TLS}_scope_1 = 0.46$	+15.51%
$L_{TLS}_scope_4 = 0.54$	+12.08%
$L_{TLS}_scope_2 = 0.54$	+5.02%
$PL = 0.778$	+4.94%
$L_{PL}_scope_1 = 0.46$	+4.03%
$L_{PL}_scope_3 = 0$	-0.21%
$L_{PL}_scope_5 = 0$	+0.20%
$L_{PL}_scope_8 = 0$	+0.18%
$L_{PL}_scope_7 = 0$	+0.17%

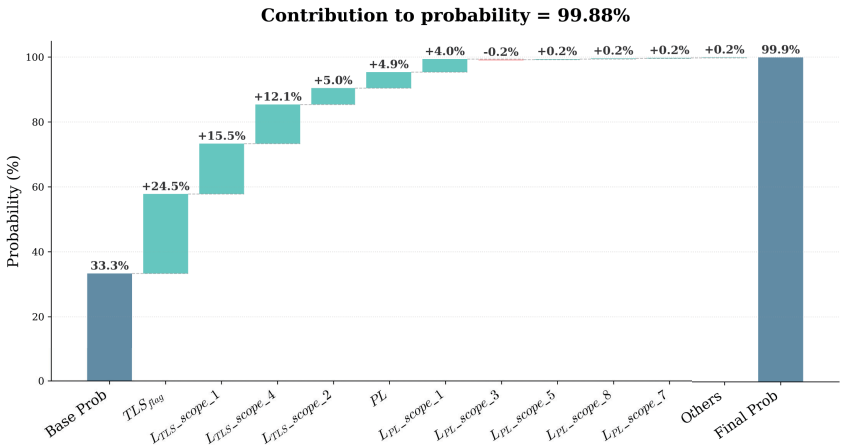


Fig. 16. Explaining a malicious test sample: On the left, a contribution table shows the value of each feature and its contribution to the model's decision, expressed as an increase or decrease in the predicted probability. On the right, a feature contribution plot displays positive contributions in green and negative contributions in red.

Feature	Effect
$TLS_{count} = 0.04$	+26.33%
$PL = 0.971$	+18.94%
$L_{TLS}_scope_8 = 0.75$	+11.18%
$L_{PL}_scope_3 = 0.015$	+4.00%
$TLS_{flag} = 0.04$	+3.80%
$L_{PL}_scope_7 = 0.005$	+1.40%
$L_{PL}_scope_1 = 0.01$	+0.78%
$L_{PL}_scope_8 = 0.96$	+0.31%
$L_{PL}_scope_2 = 0$	-0.27%
$L_{TLS}_scope_1 = 0.25$	+0.19%

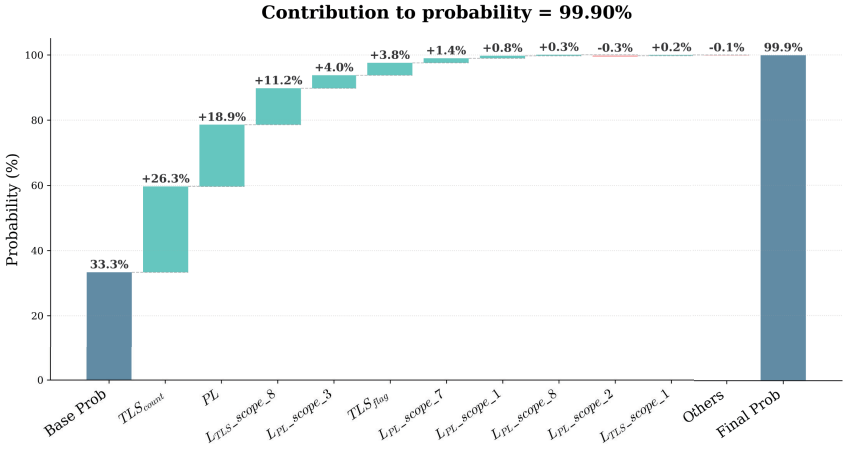


Fig. 17. Explaining a Non-DoH test sample.

in Fig. 16, the higher TLS_{flag} supports a malicious prediction. Furthermore, the more dispersed payload length distribution in this sample compared to malicious sample further increases the model's confidence in its Non-DoH classification.

VII. Conclusion

In this paper, we propose TunnelEye, a real-time and explainable malicious DoH traffic detection framework designed for high-speed backbone networks. In this framework, we construct a unidirectional traffic feature set applicable to sampled traffic, enabling detection in backbone networks with widely deployed asymmetric routing. We also design the innovative data structure, DoH-Sketch, to extract the features quickly. Experimental results show that TunnelEye significantly improves the detection accuracy in backbone network scenarios while maintaining excellent real-time performance. DoH-Sketch achieves a processing rate of 35.17 Gbps and can monitor high-speed network traffic for a long time. Even under background traffic interference, its robustness significantly outperforms similar works. In future work, we plan to extend TunnelEye to support the detection of malicious DNS over QUIC (DoQ) traffic, further broadening its applicability to emerging encrypted DNS protocols.

References

[1] Y. Wang, A. Zhou, S. Liao, R. Zheng, R. Hu, and L. Zhang, "A comprehensive survey on DNS tunnel detection," *Comput. Netw.*, vol. 197, Oct. 2021, Art. no. 108322.

[2] P. Jeitner and H. Shulman, "Injection attacks reloaded: Tunnelling malicious payloads over DNS," in *Proc. 30th USENIX Secur. Symp. (USENIX Secur.)*, 2021, pp. 3165–3182.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 22,2026 at 05:59:52 UTC from IEEE Xplore. Restrictions apply.

[3] A. Nadler, R. Bitton, O. Brodt, and A. Shabtai, "On the vulnerability of anti-malware solutions to DNS attacks," *Comput. Secur.*, vol. 116, May 2022, Art. no. 102687.

[4] F. Alharbi, Y. Zhou, F. Qian, Z. Qian, and N. Abu-Ghazaleh, "DNS poisoning of operating system caches: Attacks and mitigations," *IEEE Trans. Depend. Secure Comput.*, vol. 19, no. 4, pp. 2851–2863, Jul. 2022.

[5] EfficientIP. (2023). *Cyber Threat Intelligence: IDC 2023 Global DNS Threat Report*. [Online]. Available: https://efficientip.com/resources/cyber-threat-intelligence-idc-2023-global-dns-threat-report/

[6] J. Coker. (2021). *72% of Organizations Experienced a DNS Attack in the Past Year*. [Online]. Available: https://www.infosecurity-magazine.com/news/72-orgs-dns-attack-last-year/

[7] Z. Hu, L. Zhu, J. Heidemann, A. Mankin, D. Wessels, and P. Hoffman, "Specification for DNS over transport layer security (TLS)," Tech. Rep. RFC 7858, May 2016, doi: 10.17487/RFC7858.

[8] P. Hoffman and P. McManus, "DNS queries over HTTPS (DoH)," Tech. Rep. RFC 8484, Oct. 2018, doi: 10.17487/RFC8484.

[9] C. Huitema, S. Dickinson, and A. Mankin, *DNS Over Dedicated QUIC Connections*, document RFC 9250, 2022.

[10] J. Steadman and S. Scott-Hayward, "Detecting data exfiltration over encrypted DNS," in *Proc. IEEE 8th Int. Conf. Netw. Softwarization (NetSoft)*, Jun. 2022, pp. 429–437.

[11] R. Tang et al., "A practical machine learning-based framework to detect DNS covert communication in enterprises," in *Proc. Int. Conf. Secur. Privacy Commun. Syst.* Cham, Switzerland: Springer, 2020, pp. 1–21.

[12] C. Li, L. Dai, Z. Xu, Y. Ding, and Y. Han, "A message-based malicious detection scheme of public DNS services," in *Proc. IEEE 23rd Int. Conf. High Perform. Comput. Communications; 7th Int. Conf. Data Sci. Syst.; 19th Int. Conf. Smart City; 7th Int. Conf. Dependability Sensor, Cloud Big Data Syst. Appl. (HPCC/DSS/SmartCity/DependSys)*, Dec. 2021, pp. 733–740.

[13] J. Jia, Z. Dong, J. Li, and J. W. Stokes, "Detection of malicious DNS and web servers using graph-based approaches," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Jun. 2021, pp. 2625–2629.

[14] C. Kwan, P. Janiszewski, S. Qiu, C. Wang, and C. Bocovich, "Exploring simple detection techniques for DNS-over-HTTPS tunnels," in *Proc. ACM SIGCOMM Workshop Free Open Commun. Internet*, Aug. 2021, pp. 37–42.

[15] T. Zebin, S. Rezvy, and Y. Luo, "An explainable AI-based intrusion detection system for DNS over HTTPS (DoH) attacks," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 2339–2349, 2022.

[16] A. Aggarwal and M. Kumar, "An ensemble framework for detection of DNS-over-HTTPS (DOH) traffic," *Multimedia Tools Appl.*, vol. 83, no. 11, pp. 32945–32972, Sep. 2023.

[17] J. C. C. Pinto, E. J. Scheid, M. F. Franco, and L. Z. Granville, "Eeny, meeny, miny, moe: Analyzing and comparing the selection of DNS lookup tools," in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, Jun. 2024, pp. 1–6.

[18] A. Randall et al., "Trufflehunter: Cache snooping rare domains at large public DNS resolvers," in *Proc. ACM Internet Meas. Conf.*, Oct. 2020, pp. 50–64.

[19] J. Li et al., "DNSScope: Fine-grained DNS cache probing for remote network activity characterization," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, May 2024, pp. 1651–1660.

[20] G. C. Moura et al., "Old but gold: Prospecting TCP to engineer and live monitor DNS anycast," in *Proc. Int. Conf. Passive Act. Netw. Meas.* Cham, Switzerland: Springer, 2022, pp. 264–292.

[21] M. Lyu, H. H. Gharakheili, C. Russell, and V. Sivaraman, "Enterprise DNS asset mapping and cyber-health tracking via passive traffic analysis," *IEEE Trans. Netw. Service Manage.*, vol. 20, no. 3, pp. 3699–3716, Sep. 2023.

[22] M. E. Şahin and M. Demirci, "HELP4DNS: Leveraging the programmable data plane for effective and robust defense against DDoS attacks on DNS," *J. Netw. Comput. Appl.*, vol. 240, Aug. 2025, Art. no. 104198.

[23] Y. Afek, H. Berger, and A. Bremler-Barr, "POPS: From history to mitigation of DNS cache poisoning attacks," in *Proc. 34th USENIX Secur. Symp. (USENIX Secur.)*, 2025, pp. 3537–3556.

[24] A. M. Manasrah, T. Khdour, and R. Freehat, "DGA-based botnets detection using DNS traffic mining," *J. King Saud Univ. Comput. Inf. Sci.*, vol. 34, no. 5, pp. 2045–2061, May 2022.

[25] C. Lu et al., "An end-to-end, large-scale measurement of DNS-over-encryption: How far have we come?," in *Proc. Internet Meas. Conf.*, Oct. 2019, pp. 22–35.

[26] K. Borgolte et al., "How DNS over HTTPS is reshaping privacy, performance, and policy in the internet ecosystem," in *Proc. 47th Res. Conf. Commun., Inf. Internet Policy (TPRC)*, Washington, DC, USA, Sep. 2019, pp. 1–9.

[27] T. Böttger et al., "An empirical study of the cost of DNS-over-HTTPS," in *Proc. Internet Meas. Conf.*, Oct. 2019, pp. 15–21.

[28] C. Deccio and J. Davis, "DNS privacy in practice and preparation," in *Proc. 15th Int. Conf. Emerg. Netw. Experiments Technol.*, Dec. 2019, pp. 138–143.

[29] H. Guannan and K. Fukuda, "Characterizing privacy leakage in encrypted DNS traffic," *IEICE Trans. Commun.*, vol. E106.B, no. 2, pp. 156–165, 2023.

[30] J. Bushart and C. Rossow, "Padding ain't enough: Assessing the privacy guarantees of encrypted DNS," in *Proc. 10th USENIX Workshop Free Open Commun. Internet (FOCI)*. USENIX Association, Aug. 2020. [Online]. Available: https://www.usenix.org/conference/foci20/presentation/bushart

[31] A. Niakanlahiji, S. Orlowski, A. Vahid, and J. H. Jafarian, "Toward practical defense against traffic analysis attacks on encrypted DNS traffic," *Comput. Secur.*, vol. 124, Jan. 2023, Art. no. 103001.

[32] M. Lyu, H. H. Gharakheili, and V. Sivaraman, "A survey on DNS encryption: Current development, malware misuse, and inference techniques," *ACM Comput. Surv.*, vol. 55, no. 8, pp. 1–28, Aug. 2023.

[33] M. Zhan, Y. Li, G. Yu, B. Li, and W. Wang, "Detecting DNS over HTTPS based data exfiltration," *Comput. Netw.*, vol. 209, May 2022, Art. no. 108919.

[34] M. Moure-Garrido, C. Campo, and C. Garcia-Rubio, "Real time detection of malicious DoH traffic using statistical analysis," *Comput. Netw.*, vol. 234, Oct. 2023, Art. no. 109910.

[35] S. Ding, D. Zhang, J. Ge, X. Yuan, and X. Du, "Encrypt DNS traffic: Automated feature learning method for detecting DNS tunnels," in *Proc. IEEE Int. Conf. Parallel Distrib. Process. With Appl., Big Data Cloud Comput., Sustain. Comput. Commun., Social Comput. Netw. (ISPA/BDCloud/SocialCom/SustainCom)*, Sep. 2021, pp. 352–359.

[36] X. Liu, Y. Zhang, X. Yang, W. Gai, and B. Sun, "MFC-DoH: DoH tunnel detection based on the fusion of MAML and F-CNN," in *Proc. 21st ACM Int. Conf. Comput. Frontiers*, May 2024, pp. 267–275.

[37] S. Wu, W. Wang, and Z. Ding, "Detecting malicious DoH traffic: Leveraging small sample analysis and adversarial networks for detection," *J. Inf. Secur. Appl.*, vol. 84, Aug. 2024, Art. no. 103827.

[38] X. Du et al., "Design of an autoencoder-based anomaly detection for the DoH traffic system," in *Proc. IEEE 25th Int. Conf. Comput. Supported Cooperat. Work Design (CSCWD)*, May 2022, pp. 763–768.

[39] L. Jiao, Y. Zhu, X. Fu, Y. Zhou, F. Qin, and Q. Liu, "CCSv6: A detection model for DNS-over-HTTPS tunnel using attention mechanism over IPv6," in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, Jul. 2023, pp. 1327–1330.

[40] Canadian Institute for Cybersecurity (CIC). (2020). *CIRA-CIC-DOHBRW-2020 Dataset*. Accessed: Mar. 7, 2025. [Online]. Available: https://www.unb.ca/cic/datasets/dohbrw-2020.html

[41] S. Niktabe, A. H. Lashkari, and A. H. Roudsari, "Unveiling DoH tunnel: Toward generating a balanced DoH encrypted traffic dataset and profiling malicious behavior using inherently interpretable machine learning," *Peer Peer Netw. Appl.*, vol. 17, no. 1, pp. 507–531, Jan. 2024.

[42] B. Wang, G. Xiong, G. Gou, J. Song, Z. Li, and Q. Yang, "Identifying DoH tunnel traffic using core feathers and machine learning method," in *Proc. 26th Int. Conf. Comput. Supported Cooperat. Work Design (CSCWD)*, May 2023, pp. 814–819.

[43] R. Mitsuhashi, A. Satoh, Y. Jin, K. Iida, T. Shinagawa, and Y. Takai, "Identifying malicious DNS tunnel tools from DoH traffic using hierarchical machine learning classification," in *Proc. Inf. Secur., 24th Int. Conf. (ISC)*. Cham, Switzerland: Springer, 2021, pp. 238–256.

[44] R. Mitsuhashi, Y. Jin, K. Iida, T. Shinagawa, and Y. Takai, "Malicious DNS tunnel tool recognition using persistent DoH traffic analysis," *IEEE Trans. Netw. Service Manage.*, vol. 20, no. 2, pp. 2086–2095, Jun. 2023.

[45] Y. Qiu, B. Li, L. Jiao, Y. Zhu, and Q. Liu, "Detection of DoH tunnels with dual-tier classifier," in *Proc. 18th Int. Conf. Mobility, Sens. Netw. (MSN)*, Dec. 2022, pp. 417–421.

[46] J. Chen, H. Wu, X. Wang, S. Wang, G. Cheng, and X. Hu, "IEA-DMS: An interpretable feature-driven, efficient and accurate detection method for slow HTTP DoS in high-speed networks," *Comput. Secur.*, vol. 150, Mar. 2025, Art. no. 104291.

[47] MAWI Working Group. (2021). *MAWI Working Group Traffic Archive*. Accessed: Mar. 7, 2025. [Online]. Available: http://mawi.wide.ad.jp/mawi/

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 22,2026 at 05:59:52 UTC from IEEE Xplore. Restrictions apply.



Jiajia Du received the B.E. degree in computer science and technology from Xinjiang University, Ürümqi, China, in 2023. She is currently pursuing the master's degree with the School of Cyber Science and Engineering, Southeast University, Nanjing, China. Her research interests include cyberspace security and traffic analysis.



Songtao Liu received the B.S. degree in computer science from Jinan University, Jinan, China, in 2020, and the M.S. degree in cybersecurity from Southeast University, Nanjing, China, in 2024, where he is currently pursuing the Ph.D. degree in cybersecurity with the School of Cyber Science and Engineering. His research interests include cyberspace security and traffic analysis.



Cheng Zhi received the B.E. degree from Southeast University, Nanjing, China, in 2025, where he is currently pursuing the master's degree with the School of Cyber Science and Engineering. His research interests include cyber security and traffic analysis.



Junfan Zhao received the bachelor's degree from Guilin University of Electronic Technology, Guangxi, China, in 2024. He is currently pursuing the master's degree with the School of Cyber Science and Engineering, Southeast University. His research interests include cyberspace security and encrypted traffic analysis.



Hua Wu (Member, IEEE) received the M.S. and Ph.D. degrees from Southeast University, China, in 2000 and 2010, respectively. She is currently an Associate Professor with the School of Cyber Science and Engineering, Southeast University. Her research interests include cyberspace security, network measurement, encrypted traffic analysis, and streaming media technology.



Guang Cheng (Senior Member, IEEE) received the B.S. degree in traffic engineering from Southeast University, Nanjing, China, in 1994, the M.S. degree in computer application from Hefei University of Technology in 2000, and the Ph.D. degree in computer network from Southeast University in 2003. He is currently a Full Professor with the School of Cyber Science and Engineering, Southeast University. His research interests include network security, network measurement, and traffic behavior analysis.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 22,2026 at 05:59:52 UTC from IEEE Xplore. Restrictions apply.